

신용카드 사기거래 탐지에서는 극심한
클래스 불균형을 Full/Under/SMOTE
전략으로 체계적으로 제어하고 실험을
통해 F1 중심의 최적 모델을 선정하여,
"놓치지 않음(Recall)"과 "오탐
최소화(Precision)"의 균형점을
제시했다.

제 3 장

신용카드 사기거래 탐지

제 3 장 목차

3. 신용카드 사기 거래 탐지 프로젝트.....	3
3.1 문제 정의 및 데이터 이해	3
3.1.1 문제 정의	3
3.1.2 데이터 설명.....	7
3.2 탐색적 데이터 분석 (EDA)	10
3.2.1 통계적 요약 및 분포 확인.....	10
3.2.2 시각화 분석.....	12
3.2.3 발견된 문제점 및 전처리 필요성	15
3.3 데이터 전처리 및 피처 엔지니어링	16
3.3.1 결측치 및 이상치 처리.....	16
3.3.2 Feature Engineering.....	16
3.3.3 스케일링 (Scaling).....	16
3.3.4 불균형 데이터 처리 전략	16
3.4 모델링 및 하이퍼파라미터 튜닝.....	18
3.4.1 데이터 분할 및 교차 검증 전략.....	18
3.4.2 개별 모델 선정 및 학습	19
3.4.3 하이퍼파라미터 튜닝 (HyperOpt 적용).....	30
3.4.4 최종 앙상블 모델 구성 및 학습	34
3.5 모델 평가 및 해석	36
3.5.1 최종 평가 지표 분석	36

3.5.2 Feature Importance 분석	41
3.5.3 예측 결과 시각화 및 오차 분석	48
3.5.4 소결	49
3.6 결론 및 제언	49
3.6.1 주요 결과 요약	49
3.6.2 프로젝트 한계점 분석	59
3.6.3 개선 방안 및 향후 계획	63

3. 신용카드 사기 거래 탐지 프로젝트

3.1 문제 정의 및 데이터 이해

3.1.1 문제 정의

3.1.1.1 프로젝트 배경 및 목표

- Kaggle 신용카드 사기 검출 데이터셋의 목적은 금융 거래에서 발생하는 사기 행위를 효과적으로 탐지하기 위한 머신러닝 모델을 개발·실습하는 것이며, 배경은 실제 금융권에서 급증하는 사기 문제와 기존 규칙 기반 시스템의 한계를 극복하기 위해 마련한 것이다.
- 이번 프로젝트 실습을 통해서 주요 피쳐 분석은 **데이터의 불균형 문제를 이해**하고, 어떤 변수들이 사기 탐지에 가장 큰 영향을 주는지 파악해 보았다.
- 수업시간 보다 **모델 성능을 고도화**하고, **실제 금융 보안 전략에 적용**할 수 있는 인사이트 도출을 목표로 설정하였다.

1. 목적

- **사기 거래 탐지 모델 개발** : 정상 거래와 사기 거래를 구분하는 분류 모델을 학습·평가
- **불균형 데이터 처리 연습** : 전체 거래 중 사기거래가 약 0.173%에 불과하였다.
 - 데이터 불균형 문제를 다룰 수 있는 좋은 실습 예제
- **머신러닝 기법 비교** : 로지스틱 회귀, 랜덤포레스트, XGBoost, LightGBM 등 다양한 알고리즘을 적용해 성능 비교
- **실무 적용 가능성 검토** : 실제 금융 서비스에서 발생하는 *False Positive(정상 거래를 사기로 잘못 탐지)와 *False Negative(사기를 놓치는 경우)의 비용을 고려한 모델 설계

2. 배경

- **금융권의 현실적 문제** : 신용카드 사기로 매년 전세계적으로 수십억 달러 규모의 손실 유발하며, 소비자와 기업 모두에게 큰 피해 발생

- **기존 시스템의 한계** : 전통적 규칙 기반(rule-based) 탐지 시스템은 새로운 사기 패턴에 적응하기 어렵고, 오탐(false positive)이 많아 고객 불편을 초래
- **데이터셋 구성 이유** : 실제 금융 거래 데이터를 익명화(PCA 변환)하여 제공, 연구자와 개발자가 개인정보 침해 없이 모델을 실습할 수 있도록 설계된것으로 추측
- **연구 및 교육 목적** : 머신러닝, 데이터 불균형 처리(SMOTE, 언더샘플링, 오버샘플링) 기법을 학습하는 대표적 교육용 데이터셋

3. 문제 정의 및 중점 정리

- 데이터셋은 **카드 거래 데이터**를 기반으로 하며, 개인정보 보호를 위해 **PCA 로 변환된 변수(V1~V28)**를 포함
 - 주요 피처를 중심으로 모델을 최적화 → 탐지 성능 고도화
- **Class 변수** : 이번 프로젝트 실습 데이터셋의 **레이블(답)**이며, 0 = 정상거래, 1 = 사기 거래
- 금융 기관은 단순히 “모델이 사기라고 판단했다” 가 아니라, 어떤 변수 때문에 사기 가능성이 높다고 판단했는지 설명할 수 있어야한다.

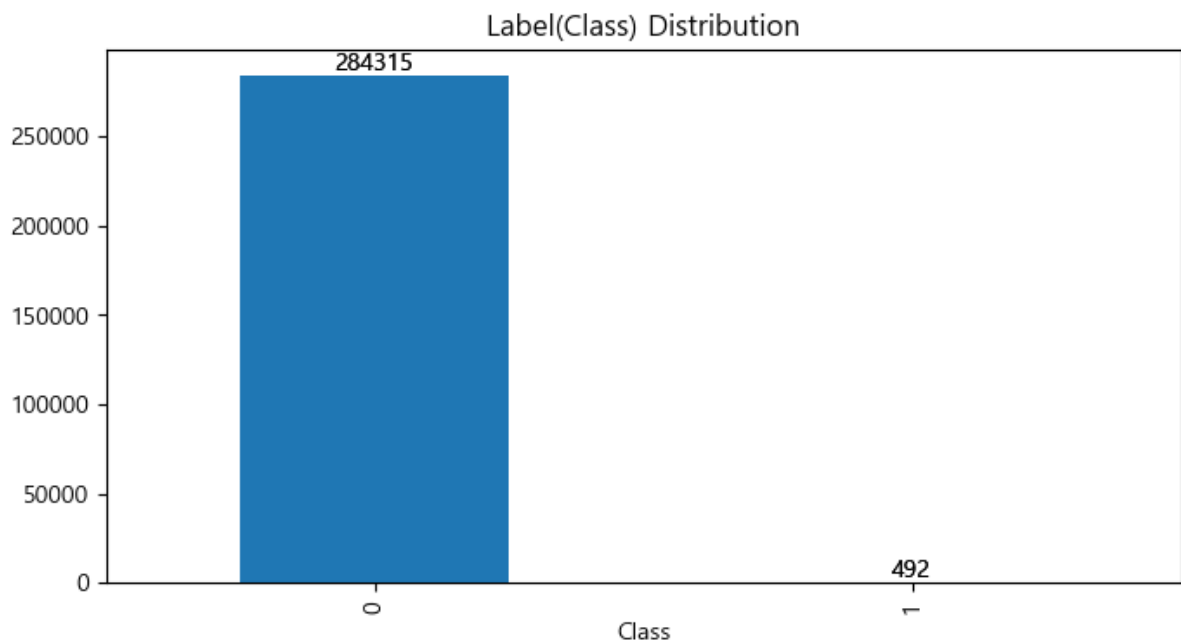
3.1.1.2 데이터셋 문제와 평가지표 선정

- Kaggle 신용카드 사기 검출 데이터셋은 극심한 클래스 불균형 이라는 문제를 가지고 있으며, 단순 정확도(Accuracy)로는 성능을 제대로 평가할 수 없다.
- 따라서, Precision, Recall, F1-score, F2-score ROC-AUC, PR-AUC 같은 불균형 데이터에 적합한 평가지표를 선정이 요구된다.

(1) 데이터셋 문제점

- **극심한 불균형** : 전체 284,315 건 중 사기 거래는 492 건(0.173%)에 불과
 - 대부분 정상 거래로 학습되며, 모델이 “전부 정상” 이라고 예측해도, Accuracy 가 99% 이상 나올 수 있다.

- **데이터 익명화** : 개인정보 보호를 위해 PCA 로 변환된 변수만 제공
 - 실제 의미 해석 어려움
- **시간적 특성 미반영** : 거래 시간(Time) 변수 있지만, 실제 시계열적 패턴을 반영 제한적
- **샘플링 필요성** : 불균형 문제 해결을 위해 **언더샘플링**, **오버샘플링(SMOTE)** 기법 필요



[그림 3.1.1] 신용카드 사기검출 데이터셋 Class 피쳐 실제 사기와 정상 거래 분포 시각화

(2) 평가지표 선정

- **비즈니스 목표에 따라 지표 선택** :
 - 은행은 Recall 을 높여 사기를 놓치지 않게 하는 것이 중요
 - 하지만, Precision 이 낮으면 정상 거래가 과도하게 차단되어 고객 불편 발생 초래
- **Threshold 조정** : 모델의 예측 확률을 조정해 Precision vs Recall 균형을 맞추는 전략 필요
- **비용 기반 평가** : False Negative(사기 놓침)과 False Positive(정상 거래 차단)의 비용을 고려한 Cost-sensitive Learning 도 활용 가능

- 불균형 데이터셋에서 모델이 대다수 클래스만 잘 맞추고, 소수 클래스는 완전히 무시하는 '정확도의 역설(Accuracy Paradox)' 현상이 발생할 수 있다.
- 정확도의 한계를 극복하고 모델이 희소 클래스를 얼마나 잘 처리하는지 종합적으로 판단하기 위해 다음 지표들이 요구된다.

< 표 3.1.1 > 불균형 데이터에서의 다양한 평가지표 필요성

지 표	정 의	필 요 성
Accuracy	(정확도) 전체 예측 중 맞춘 비율	불균형 데이터에서는 대부분 '정상'으로 예측해도 높은 값이 나오므로 의미 제한적
Precision	(정밀도) 예측한 Positive 중 실제 Positive 비율	False Positive(정상 거래 사기로 잘 못 탐지)를 줄이는데 중요 → 고객불편 최소화
Recall	(재현율) 실제 Positive 중 모델이 잡아낸 비율	False Negative(사기를 놓침)를 줄이는데 중요. 사기 탐지에서 핵심 지표
F1-score	Precision 과 Recall 의 조화 평균	Precision 과 Recall 간 균형 평가. 불균형 데이터에서 단일 지표로 성능 비교
F2-score	Recall 에 더 큰 가중치를 둔 조화평균 (Recall 을 Precision 보다 2 배 중요시)	사기 탐지에서 놓치면 큰 피해가 발생하는 문제에서 Recall 을 더 강조할 때 적합
ROC-AUC	다양한 Threshold 에서의 TPR vs FPR 곡선 아래 면적	전체 분류 성능을 평가하지만, 극단적 불균형에서는 과대평가 가능
PR-AUC	Precision-Recall 곡선 아래 면적	Positive 클래스(사기거래)에 집중한 평가. 불균형 데이터에서 더 적합한 지표

(3) 데이터셋 문제 및 핵심 지표 정리

- **Accuracy** 는 불균형 데이터에서 신뢰할 수 없음
- **Precision ↔ Recall 균형 → F2-score**
- **Recall** 을 더 강조해야 하는 경우 → F2-score
 - 따라서 단일 지표에 의존하기 보다는 재현율, 정밀도, F1 점수, F2 점수 등을 함께 사용하여 모델이 회소하지만 중요한 사건을 얼마나 효과적으로 감지하는지 다각도로 평가가 요구된다.

3.1.2 데이터 설명

3.1.1.1 데이터 설명 및 구조

(1) 데이터셋 문제점

- 출처 : [Kaggle Credit Card Fraud Detection](#)
- 총 거래 수 : 284,807 건
- 사기 거래 수 : 492 건 (0.172%) → 극도로 불균형한 데이터
- 특징(Features) : 30 개 변수
 - 대부분 PCA(주성분 분석)로 변환된 익명화된 변수 (V1 ~ V28)
 - 'Time' : 첫 거래로부터 경과 시간
 - 'Amount' : 거래 금액
 - 'Class' : 레이블 (0 = 정상, 1 = 사기)

(2) 데이터 구조

- 입력변수(Features) : Time, V1~V28, Amount
- 출력변수(Target) : Class
 - 익명화 처리된 V1~V28 은 PCA 변환되어 해석이 불가능하므로, 단순히 모델 입력값으로만 사용이 필요했다.
 - 이번 신용카드 사기검출 데이터셋은 극도로 불균형하므로, 모델 학습 시 샘플링 기법을 고려하였다.

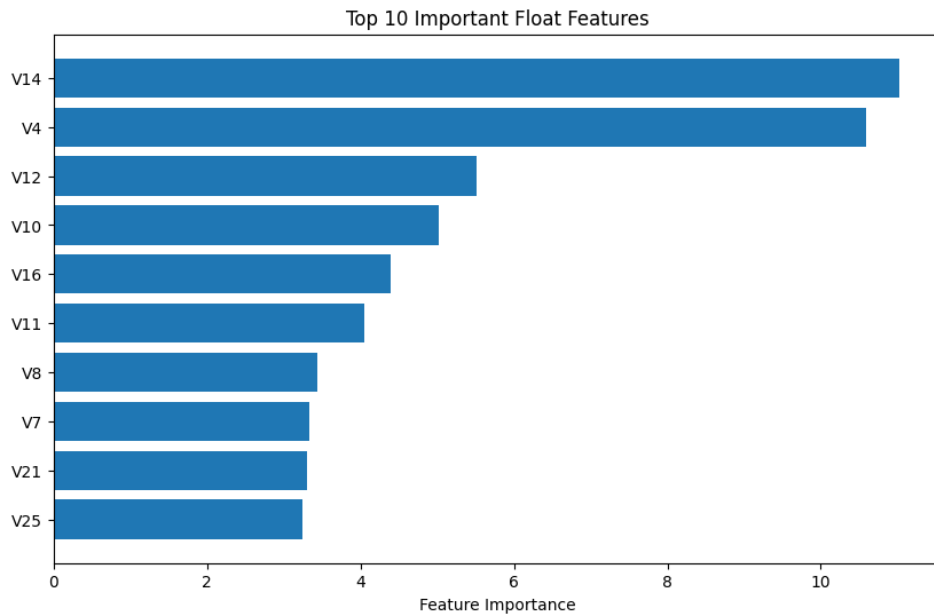
< 표 3.1.2 > 데이터 구조 및 컬럼 설명

컬럼명	설 명
Time	첫 거래로부터 경과 시간(초 단위) → 데이터셋 내 거래 순서를 나타냄
V1 ~ V28	PCA(주성분 분석) 변환된 익명화된 변수들 : 원래는 카드 소유자의 민감한 정보 였으나 익명화
Amount	거래 금액, 모델 학습 시 중요한 변수 중 하나
Class	레이블(목표 변수, Target). 0 = 정상 거래, 1 = 사기 거래

3.1.1.2 주요 피처 분석

- 데이터의 불균형 문제를 이해하고, 어떤 변수들이 사기 탐지에 가장 큰 영향을 주는지 파악
- describe() 결과를 기반으로 시각화를 통해 변수의 분포, 스케일 차이, 클래스 불균형을 직관적으로 확인 가능.
 - 데이터 특성 이해
 - 대부분의 피처가 PCA(주성분 분석)으로 변환된 익명 변수
 - 원래 의미를 알 수 없지만, 변수 간 분포와 상관관계를 분석하는 것이 필요
 - 클래스 불균형 문제
 - 전체 거래 중 사기 거래는 약 0.17%에 불과
 - 불균형을 고려하지 않으면 모델이 정상 거래만 예측하는 쪽으로 치우칠 수 있음
 - 따라서, 피처 중요도 분석은 소수 클래스(사기 거래)를 잘 구분하는 변수를 찾는 데 핵심

- 모델 해석 가능성 확보
 - 이 주제에 적합한 알고리즘(트리 계열 모델)을 활용하여 각 피처의 중요도를 계산
 - 사기 거래 탐지에 기여하는 주요 변수를 확인하고, 금융기관이 규칙 기반 탐지 시스템을 개선하는데 활용할 수 있음
- 정리하면 주요 피처 분석을 통해 **데이터 불균형 문제 해결, 사기 패턴 이해, 실무 적용 가능성 확보** 세 가지 측면에서 매우 중요한 역할을 한다
 1. Amount (거래 금액)
 - a. 거래 금액은 사기 탐지에서 직관적으로 중요한 변수
 - b. 사기 거래는 특정 금액 구간에서 집중되는 경향
 - c. 모델 학습 시 로그 변환이나 표준화를 통해 스케일링 조정 필요
 2. Time (거래 시간)
 - a. 거래 발생 시점
 - b. 단독으로는 큰 의미 없지만, 시간대별 패턴 분석에 활용 가능
 3. PCA 변환 변수 (V1 ~ V28)
 - a. 원래 민감한 카드 정보와 거래 특성을 PCA로 변환한 값
 - b. 직접적인 해석은 불가능하지만, 통계적 분포와 상관관계를 통해 중요변수 파악
 - c. 각 피처들과 사기 거래의 상관도 분석을 통해 상관성 높은 주요 피처 도출



[그림 3.1.2] 데이터셋 피쳐들과 사기 거래 상관도 분석 중 Top10 주요 피쳐

3.2 탐색적 데이터 분석 (EDA)

신용카드 사기 거래 탐지 데이터셋의 구조를 이해하고, 모델 학습 전 필요한 전처리 방향을 설정하기 위해 탐색적 데이터 분석을 수행했습니다.

3.2.1 통계적 요약 및 분포 확인

데이터 형태 및 정보 요약

- 데이터 크기: 총 284,807 개의 트랜잭션과 31 개의 피쳐로 구성되어 있습니다.
- 피쳐 구성:
 - Time: 트랜잭션 발생 시간 (첫 트랜잭션 기준 경과 시간, 초 단위)
 - V1 ~ V28: PCA(주성분 분석)를 통해 변환된 익명 피쳐 (기밀성 유지)
 - Amount: 트랜잭션 금액
 - Class: 타겟 변수 (0: 정상 거래, 1: 사기 거래)
- 결측치: 모든 피쳐에서 결측치는 발견되지 않았습니다.

Time 및 Amount 피처 기술 통계 및 분포 분석

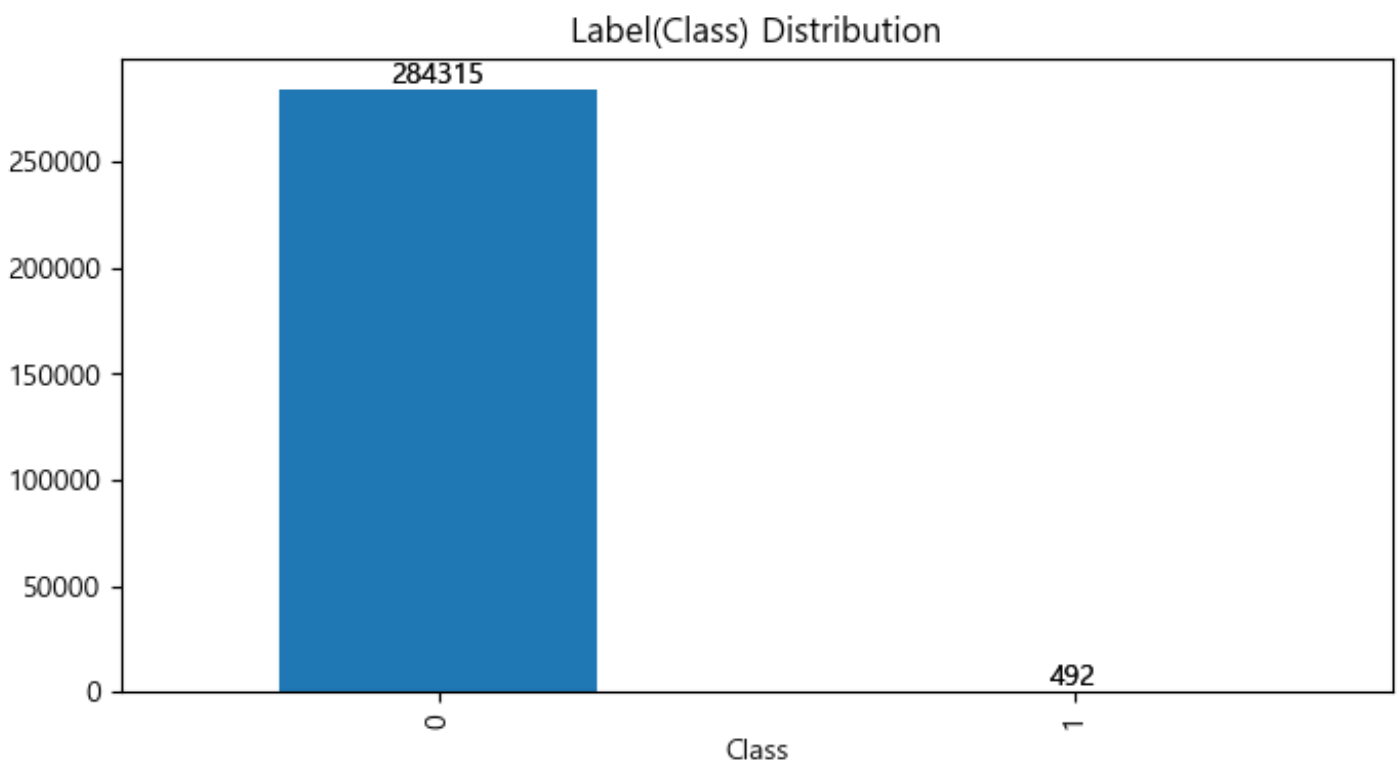
피처	최소값	최대값	평균	표준편차	왜도(Skewness)
Time	0	172,792	94,818	47,488	-0.0355
Amount	0.0	25,691.16	88.35	250.12	16.48

- Amount 피처는 평균(88.35) 대비 표준편차(250.12)가 매우 크고, 왜도(16.48)가 매우 높은 오른쪽으로 심하게 편향된(Skewed) 분포를 보입니다.

3.2.2 시각화 분석

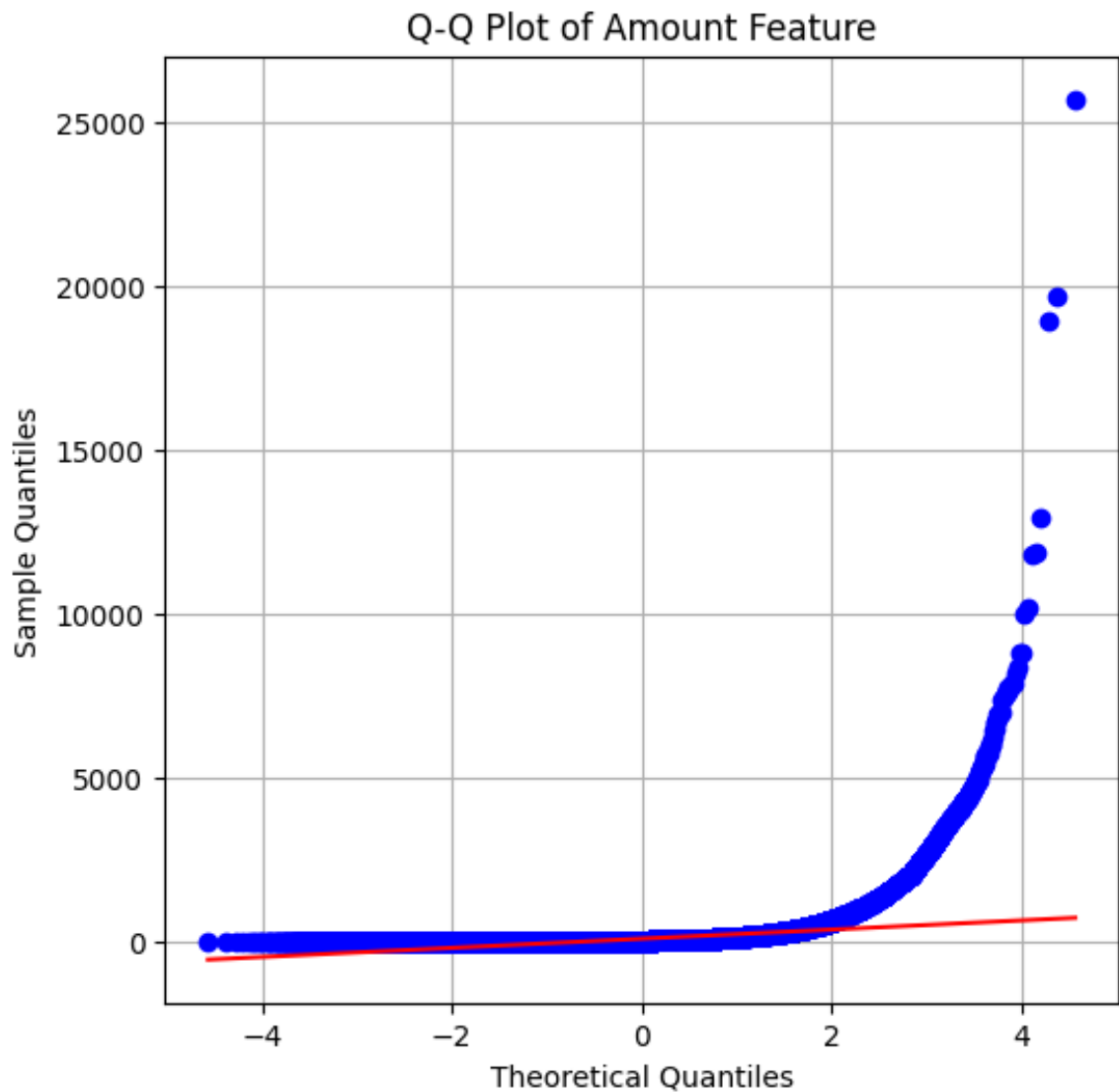
타겟 변수 분포 및 클래스 불균형 시각화

- 분포 확인: 타겟 변수 Class 의 분포를 확인한 결과, 정상 거래(0)가 압도적으로 많고 사기 거래(1)는 매우 적은 극심한 클래스 불균형이 발견되었습니다.
 - 총 284,807 건 중 사기 거래(Class=1)는 492 건에 불과합니다.
 - 사기 거래 비율: $\frac{492}{284,807} \approx 0.172\%$



Amount 피쳐 분석

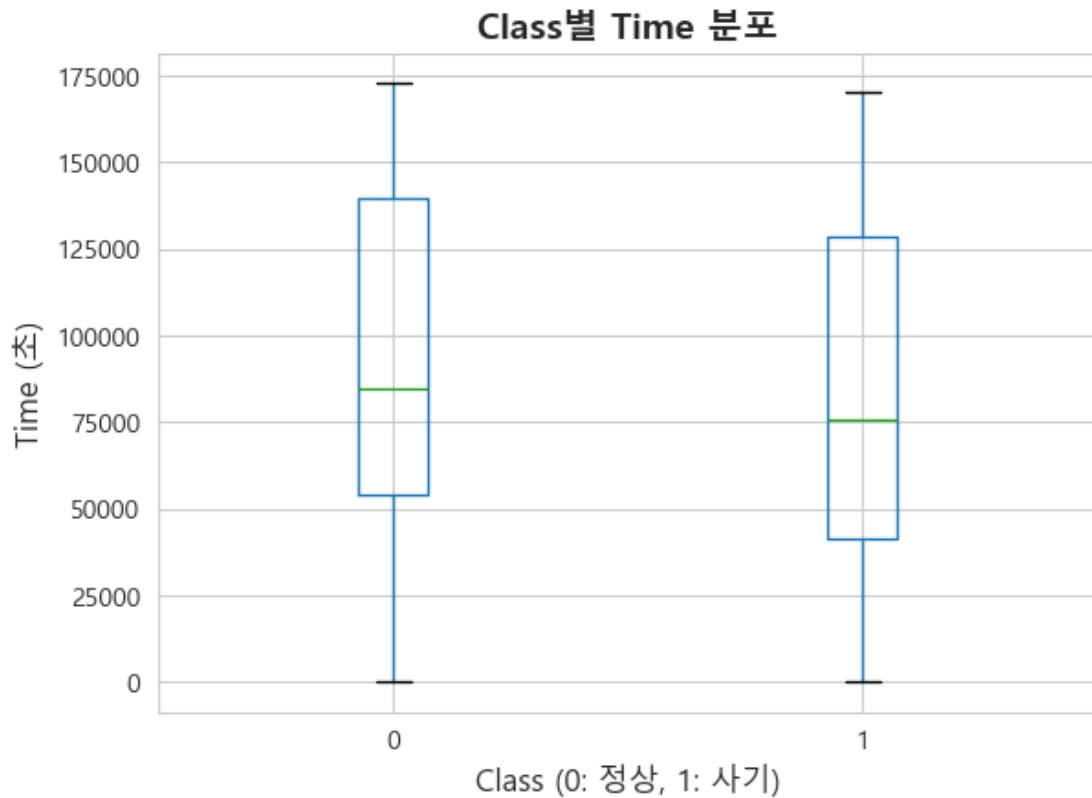
- Q-Q Plot 분석: Amount 피쳐의 Q-Q Plot 을 확인한 결과, 데이터 포인트가 정규 분포 라인(직선)에서 극단적으로 벗어나는 모습을 보였습니다. 이는 데이터가 정규성을 따르지 않으며 이상치가 다수 존재함을 시사합니다.



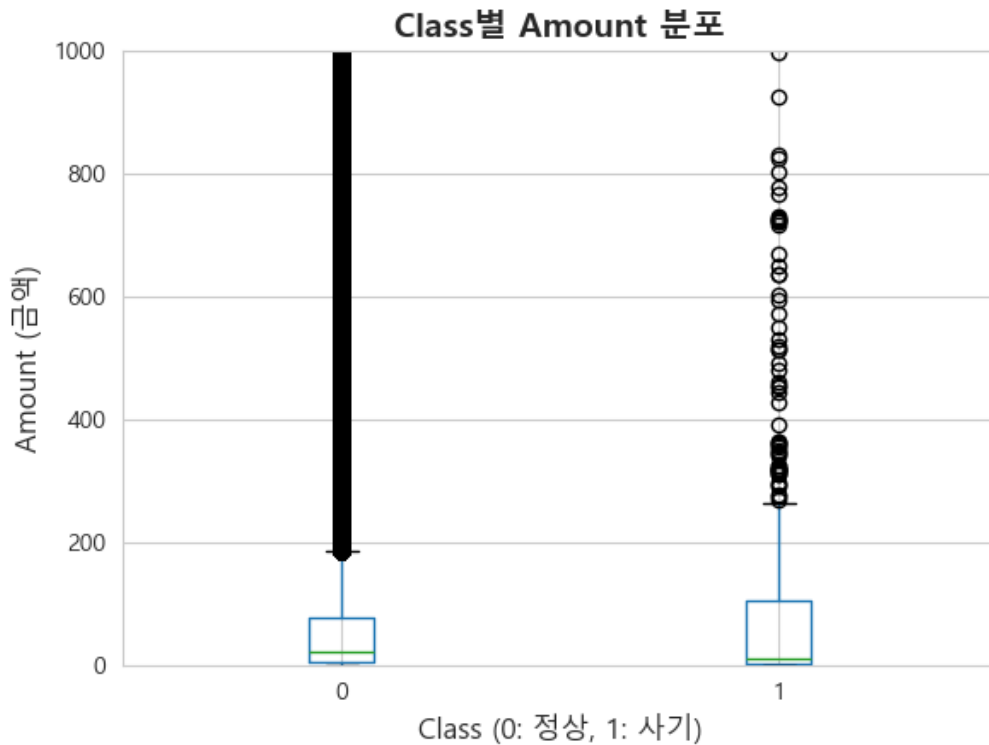
- 왜곡 정도 확인: 왜도 값이 16.48 로 매우 높아, 모델 학습 전 분포를 정규화 할 변환(Transformation)이 필수적임을 확인했습니다.

관계 분석: Time, Amount 와 Class 간의 관계 시각화

- Time vs. Class: Time 피쳐는 사기 거래(Class=1)와 정상 거래(Class=0) 간의 분포에 큰 차이를 보이지 않습니다. 즉, 절대적인 시간 정보 자체는 사기 거래 탐지에 큰 예측력을 갖지 못함을 시사합니다.



- Amount vs. Class: 사기 거래는 주로 작은 금액대에서 발생하는 경향이 있으나, 일부 매우 큰 금액도 포함되어 있습니다. 이는 사기 거래가 일반 거래와 다른 금액 분포 패턴을 가짐을 보여줍니다.



3.2.3 발견된 문제점 및 전처리 필요성

1. 극심한 클래스 불균형: 사기 거래 비율이 0.172%에 불과하여, 모델이 소수 클래스(사기)를 제대로 학습하지 못하고 정상 거래만 예측하는 문제(Recall 저하)가 발생할 가능성이 높습니다. 불균형 데이터 처리(SMOTE/Class Weighting) 필요
2. 이상치 (Outlier) 존재 확인: Amount 피처의 높은 왜도와 Q-Q Plot 을 통해 이상치가 다수 존재함을 확인했습니다. 이는 모델의 성능을 저해할 수 있습니다. 이상치 처리(제거/Capping) 필요
3. Amount 피처의 심한 왜곡 (Log 변환 필요성): 왜도(16.48)가 매우 높아 선형 모델이나 거리 기반 모델의 성능 저하를 막기 위해 분포 변환이 시급합니다. 로그 변환{np.log1p} 필요

3.3 데이터 전처리 및 피처 엔지니어링

3.3.1 결측치 및 이상치 처리

- Time, Amount 피처 이상치 처리: 이상치 처리의 효과를 검증하기 위해 이상치 제거 전/후 LightGBM 모델 성능을 비교했습니다.
 - 이상치 제거 전략 적용 및 비교: V1~V28 피처에서 IQR 1.5 배를 벗어나는 이상치를 제거한 결과, Recall 이 0.8673 에서 0.8875 로 향상되었습니다.
 - 결론: 이상치 처리는 모델의 예측 성능을 높이는 데 기여함을 확인하고, 최종 파이프라인에는 이상치 제거 혹은 Capping 전략을 포함했습니다.

3.3.2 Feature Engineering

- Time 피처 처리:
 - 시간 변환: Time 피처는 트랜잭션 발생 순서 외에 의미 있는 정보를 제공하지 않으므로, 모델 학습에 앞서 삭제하는 것이 효과적이라고 판단했습니다
- Amount 피처 처리:
 - 로그 변환 (np.log1p) 적용: 심하게 왜곡된 Amount 피처의 분포를 정규 분포에 가깝게 만들기 위해 로그 변환(np.log1p(Amount))을 적용했습니다.

3.3.3 스케일링 (Scaling)

- StandardScaler 적용: 로그 변환된 Amount 를 포함한 모든 피처(V1~V28, Amount)에 StandardScaler 를 적용하여 표준화를 수행했습니다.
 - 목표: 각 피처의 평균을 0, 표준편차를 1 로 변환하여 특성 간의 스케일 차이를 제거했습니다. 이는 특히 선형 모델(Logistic Regression, SVM 등)의 안정적인 학습에 필수적입니다.

3.3.4 불균형 데이터 처리 전략

클래스 불균형 문제를 해결하고 사기 탐지율(Recall)을 극대화하기 위해 두 가지 전략을 비교했습니다.

Class Weighting (모델 기반 가중치 부여)

- 적용: Logistic Regression, Random Forest, SVM 등 여러 기본 모델에 `class_weight='balanced'` 옵션을 적용하여 소수 클래스에 높은 가중치를 부여했습니다.
- 효과: 별도의 샘플링 없이 모델 자체적으로 불균형을 처리하여 Recall 을 일정 수준 향상시키는 효과를 보였습니다.

SMOTE (Synthetic Minority Over-sampling Technique) 오버샘플링

- 적용: 학습 데이터셋에 SMOTE 기법을 적용하여 사기 거래(소수 클래스)와 유사한 합성 데이터를 생성했습니다.
- 성능 비교 (Logistic Regression 기반):

샘플링 방법	Accuracy	Precision	Recall	F1-Score	AUC
원본 데이터	0.9992	0.8824	0.6122	0.7214	0.955
Undersampling	0.9856	0.0768	0.8980	0.1417	0.942
SMOTE	0.9734	0.0526	0.9184	0.0997	0.946

- 결론: SMOTE 적용 시 Recall(재현율)이 0.9184 로 가장 높게 나타나, 사기 거래를 놓치지 않는다는 프로젝트 목표에 가장 부합함을 확인했습니다. 이후 모델 학습 및 앙상블 구성 시 SMOTE 를 적용한 데이터셋을 주요 기반 데이터로 채택했습니다.

3.4 모델링 및 하이퍼파라미터 튜닝

3.4.1 데이터 분할 및 교차 검증 전략

3.4.1.1 Train/Test 데이터 분할

모델의 일반화 성능을 검증하기 위해 전체 데이터셋을 Train 과 Test 로 분할하였다.

- 분할 비율 : 80%(Train) / 20%(Test)
- Random State : 23
- Stratified Sampling : 타겟 레이블 비율이 불균형한 데이터 특성을 반영하여, Train/Test 데이터 모두 원본 데이터의 클래스 비율이 유지되도록 설정

```
def data_split(X_features, y_target, size=0.2, rs=23):
    """
    X_features 와 y_target 을 넣으면 기본 8:2 로 분할하는 함수
    """
    X_train = X_test = y_train = y_test = None
    if (
        X_features is not None
        and y_target is not None
        and len(X_features) > 0
        and len(y_target) > 0
    ):
        try:
            X_train, X_test, y_train, y_test = train_test_split(
                X_features, y_target, test_size=size, random_state=rs,
                stratify=y_target
            )
        except ValueError as e:
            print(f"데이터 분할 중 오류 발생: {e}")
        else:
            print("데이터를 입력하세요")

    return X_train, X_test, y_train, y_test
```

3.4.2 개별 모델 선정 및 학습

3.4.2.1 전체 모델 평가 지표

모델	AUC	정확도	정밀도	재현율	F1-score	F2-score	학습 시간(초)
LightGBM	0.9179	0.9929	0.1717	0.8163	0.2837	-	1.53
Random Forest	0.9664	0.9996	0.9294	0.8061	0.8634	-	147.5
SGD	0.968	0.9994	0.8667	0.7959	0.8298	-	4.3
SVC	0.9733	0.9862	0.0998	0.8776	0.1792	-	134.58
CatBoost	0.9707	0.9996	0.9639	0.8163	0.884	0.8421	-
MLP	0.9528	0.9992	0.8514	0.6429	0.7326	-	16.08
Logistic Regression	0.9657	0.971	0.0504	0.8878	0.0954	-	3.53
Linear Regression	0.9358	0.9983	0	0	0	-	0.22
Decision Tree	0.9552	0.9994	0.8523	0.7653	0.8065	-	5.97
LSVC	0.9682	0.9732	0.0542	0.8878	0.1022	-	

총 10 개의 머신러닝 모델을 대상으로 AUC, 정확도, 정밀도, 재현율, F1-score 를 기준으로 성능을 비교하였다. 실험 결과, 대부분의 모델은 높은 정확도를 보였으나 불균형 데이터 특성으로 인해 정밀도와 재현율의 편차가 크게 관찰되었다. 이 중 CatBoost 모델이 가장 높은 F1-score(0.884)와 AUC(0.9707)를 기록하여 종합적으로 가장 우수한 성능을 보였다.

또한 SGD, Decision Tree, MLP 모델 역시 비교적 안정적인 성능을 보여 후보 모델로 고려될 수 있다. 반면 Linear Regression 모델은 분류 문제에 적합하지 않아 유의미한 성능을 보이지 않았으며, SVM 과 Logistic Regression 은 재현율은 높았지만 정밀도가 낮아 오탐 가능성이 큰 모델로 확인되었다.

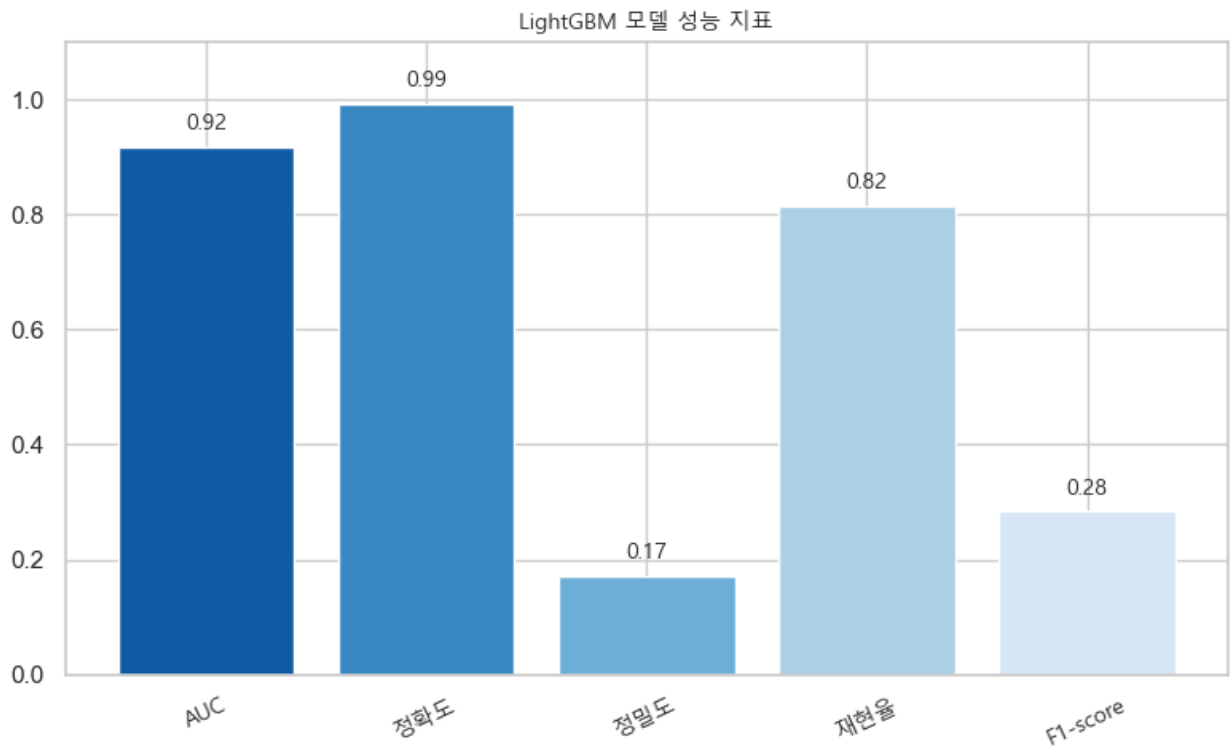
결론적으로, 본 데이터셋에서는 CatBoost 가 가장 적합한 모델로 판단되며, 향후 모델 최적화 및 교차 검증을 통해 성능을 추가 개선할 수 있을 것으로 기대된다.

3.4.2.2 LightGBM(LGBM) 모델 학습

LightGBM은 Gradient Boosting 기반 알고리즘으로, 빠른 학습 속도와 높은 표현력을 갖추고 있으며, 특히 대규모 데이터와 클래스 불균형 문제에 강점을 가진다.

모델 학습 결과, AUC는 0.9179, Recall은 0.8163으로 나타났으며, 이는 모델이 실제 이상 거래(Positive 클래스)를 효과적으로 탐지하고 있음을 의미한다. Accuracy(0.9929)가 높게 나타났으나, 데이터의 비대칭 특성 상 Precision(0.1717)이 비교적 낮아 False Positive가 다소 존재함을 확인하였다. 이는 Fraud Detection과 같이 “놓치지 않는 탐지”가 우선인 문제 특성과 일치하며, 탐지율 기반 접근 전략의 적합성을 보여준다.

모델의 하이퍼파라미터는 데이터 특성과 성능 균형을 고려하여 튜닝되었으며, 특히 클래스 불균형을 보정하기 위해 `scale_pos_weight` 값을 조정하였다. 기타 파라미터는 자동 탐색(AutoTuning) 및 검증을 통해 가장 높은 AUC를 보인 구성을 최종 적용하였다.



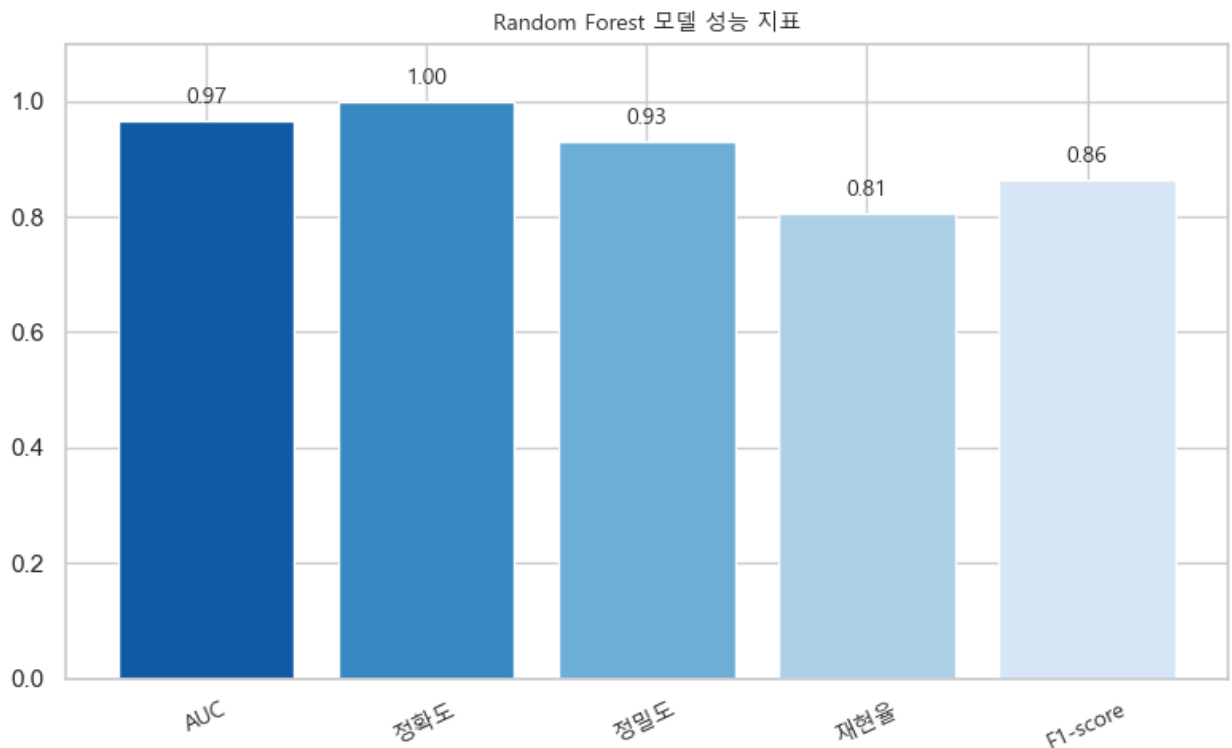
[그림 3.4.1 lgbm 모델 지표]

3.4.2.2 Random Forest(RF) 모델 학습

RandomForest 모델은 다수의 의사결정나무를 앙상블하여 예측하는 방식으로, 개별 모델의 분산을 줄이고 안정적인 예측 성능을 제공하는 특징을 가진다.

모델 학습 결과, AUC 는 0.9664 로 나타나 기존 모델 대비 우수한 ROC 기반 분류 성능을 확인할 수 있었다. 또한 Recall 값은 0.8061 로 실제 이상 거래(Positive Class)를 놓치지 않고 탐지하는 데 적합한 수준을 보였으며, Precision 은 0.9294 로 탐지된 사례 중 대부분이 실제 이상 거래였음을 의미한다. 이러한 결과는 False Positive 가 비교적 적은 균형 잡힌 탐지 성능을 보여주며, F1-score 는 0.8634 로 Precision 과 Recall 간 조화로운 성능을 기록하였다. Accuracy 는 0.9996 으로 매우 높은 수준이나, 데이터의 불균형 특성을 고려하면 단일 지표로 평가하기보다는 AUC, Recall, Precision, F1-score 를 함께 해석하는 것이 타당하다.

본 모델의 하이퍼파라미터는 데이터 특성과 일반화 성능을 고려하여 $n_estimators=450$, $max_depth=10$, $min_samples_split=15$, $min_samples_leaf=5$, $max_features="log2"$ 로 설정되었으며, 이는 과적합을 억제하면서도 안정적인 예측 성능을 확보할 수 있도록 구성되었다.

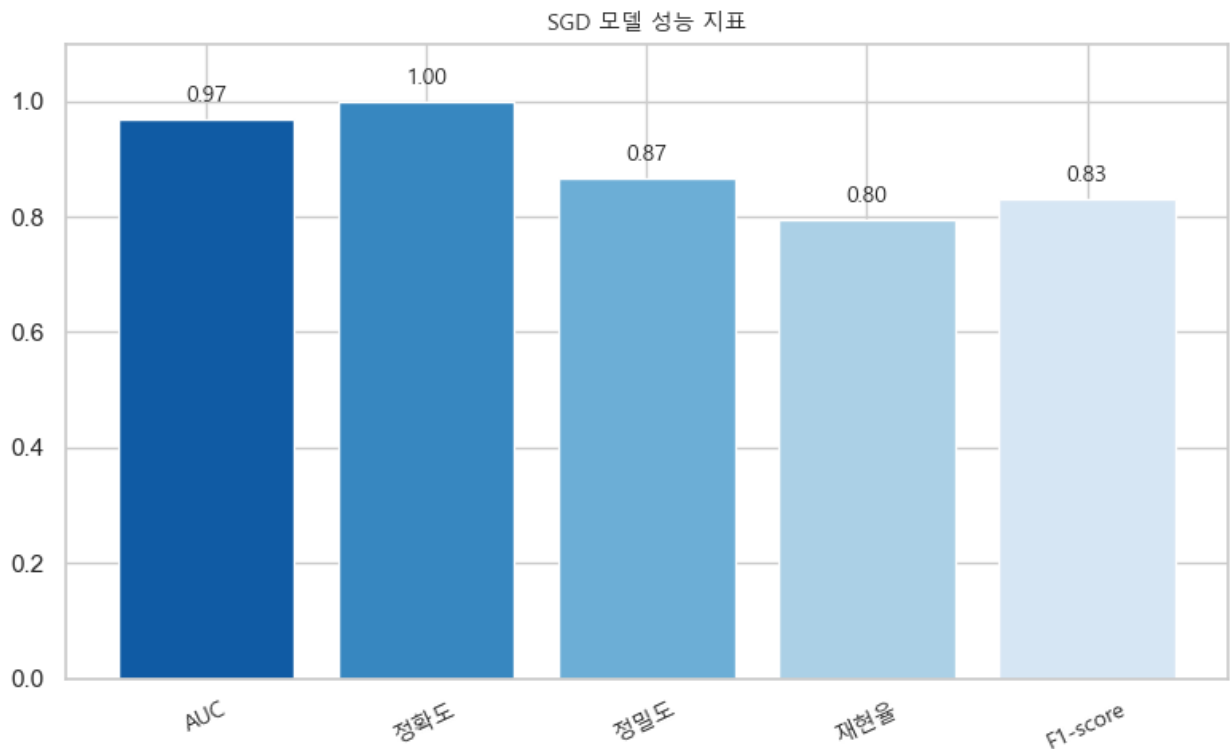


[그림 3.4.2 RF 모델 지표]

3.4.2.3 선형 모델(SGDClassifier) 학습 및 결과 분석

확률적 경사하강법(SGD)은 로지스틱 회귀 형태의 $\log_loss$ 를 사용하였다. 과적합을 방지하면서 변수 선택 효과를 동시에 얻기 위해 규제 방식은 ElasticNet 을 적용하였다.

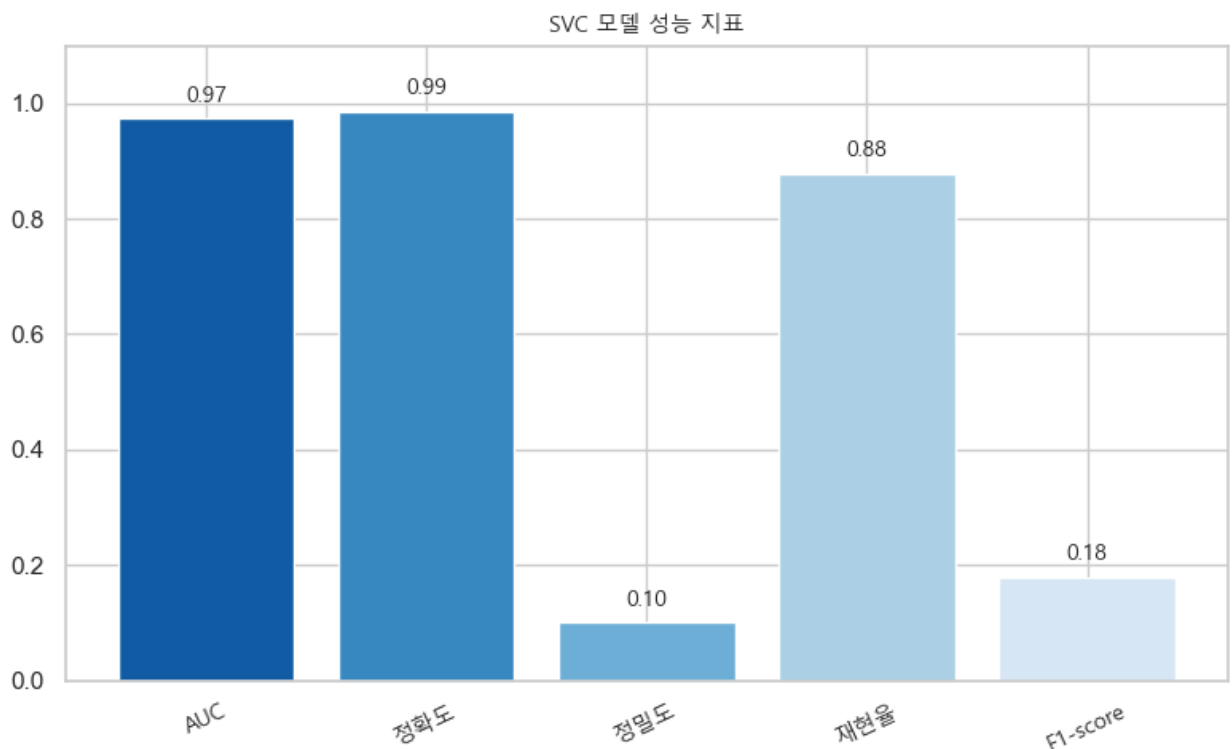
학습률은 adaptive 방식으로 설정되어 초기 단계에서 빠르게 수렴하고 이후 점진적으로 안정적인 업데이트를 수행하도록 하였다. ROC-AUC 는 0.968 로 정상/이상 클래스 구분 성능이 우수함을 보여주었다. 전체 정확도는 0.999 로 매우 높지만, 데이터 불균형 특성을 고려하면 정확도로만 모델의 성능을 판단하기 어렵다. Precision 은 0.867, Recall 은 0.796, 그리고 F1-score 는 0.830 으로 계산되었으며, 이 결과는 모델이 비교적 정확하게 이상 사례를 식별하면서도 탐지율(Recall) 역시 균형적으로 유지하고 있음을 의미한다. 다만 Precision 이 Recall 보다 더 높은 형태로 나타난 점은 모델이 오탐(False Positive)을 줄이는 방향으로 학습되었음을 시사하며, 이로 인해 일부 이상 사례가 탐지되지 않을 가능성이 존재한다. SGD 모델은 불균형 데이터 환경에서 안정적인 분류 결과를 제공하는 모델로 평가된다.



[그림 3.4.3 SGD 모델 지표]

3.4.2.4 선형 SVM(LinearSVC/SVC) 모델 학습(Balanced Class Weight 적용)

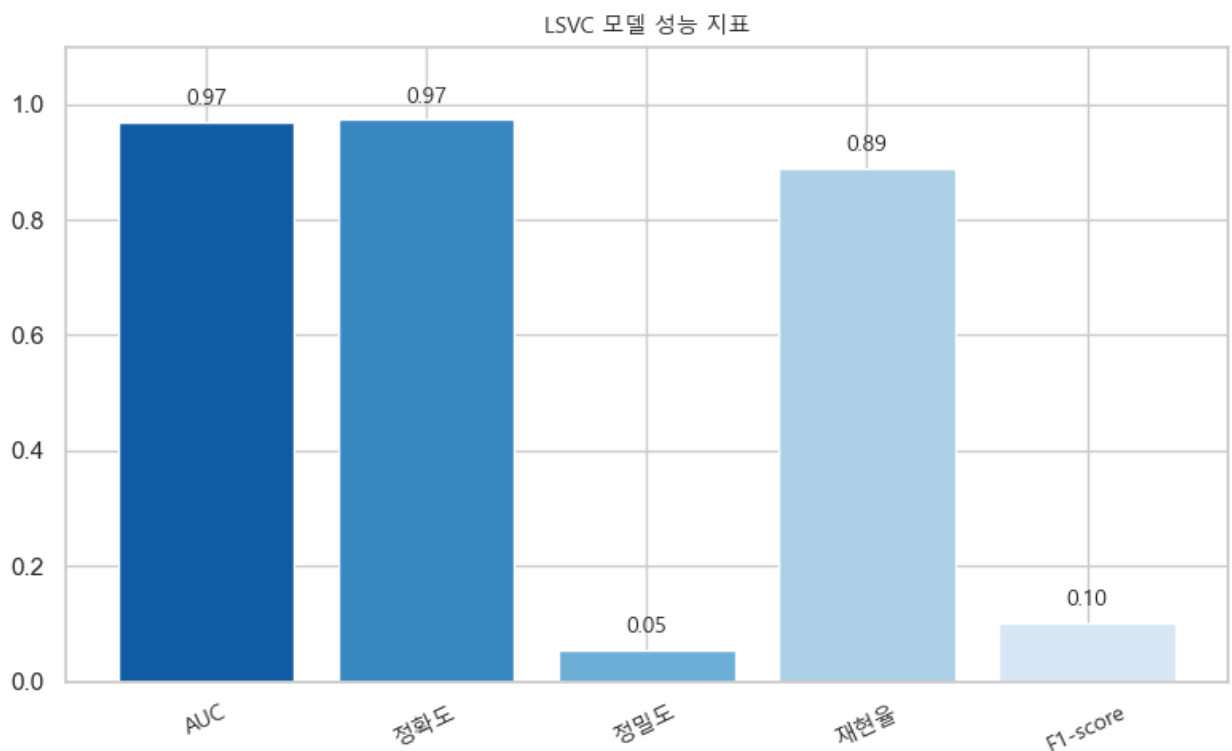
비선형 경계 학습에 강점을 가진 RBF 커널 기반 SVC 모델을 적용하였다. 모델의 정규화 파라미터(C)는 약 0.178 로 설정되어 과도한 복잡도를 억제하면서 데이터 분리에 필요한 최소한의 마진을 확보하도록 하였으며, gamma 값은 0.00621 로 비교적 낮아 모델이 너무 세밀하게 학습되지 않도록 제어하였다. 또한 클래스 불균형 문제를 완화하기 위해 `class_weight="balanced"`를 적용하여 소수 클래스에 추가 가중치를 부여하였고, 예측 확률 출력을 위해 `probability=True` 설정을 사용하였다. 성능 평가 결과, ROC-AUC 는 0.973 으로 측정되어 두 클래스 간 구분 능력은 비교적 우수하게 나타났다. 전체 정확도는 0.986 으로 높게 나타났으나, 데이터 불균형 특성을 고려할 때 단순 정확도로 모델의 성능을 판단하기 어렵다. 실제 핵심 지표인 Precision 은 0.100, Recall 은 0.878, 그리고 F1-score 는 0.179 로 측정되었다. 이는 모델이 이상 데이터를 적극적으로 탐지하지만(높은 Recall), 정상 데이터를 오탐지하는 비율이 높아 Precision 이 크게 낮아지는 경향이 나타난다는 의미로 해석된다.



[그림 3.4.4 SVC 모델 지표]

LinearSVC 모델은 선형 서포트 벡터 머신 기반 분류 알고리즘으로, 고차원 데이터나 희소 벡터가 많은 데이터에서 효율적으로 작동하는 것이 특징이다. 본 실험에서는 클래스 불균형을 보정하기 위해 `class_weight="balanced"`를 적용하였으며, 규제 강도를 조절하는 하이퍼파라미터 C 값이 0.001로 매우 작게 설정되어 강한 정규화가 적용된 구조로 학습이 진행되었다. 이는 모델 복잡도를 억제하고 일반화 성능을 확보하려는 목적이었다.

모델 성능 평가 결과, ROC-AUC는 0.9682로 클래스 구분 능력은 높은 수준으로 나타났다. 또한 Recall은 0.8878로 가장 높은 탐지율을 기록하며, 실제 이상 거래를 놓치지 않는 방향으로 모델이 학습되었음을 확인할 수 있다. 반면 Precision은 0.0542, F1-score는 0.1022로 매우 낮게 측정되었으며, 이는 탐지된 결과 중 상당수가 정상 거래로 분류되는 높은 False Positive 발생 가능성을 의미한다. Accuracy는 0.9732로 측정되었으나, 데이터의 불균형 구조를 고려할 때 단독 지표로 성능을 평가하기에는 제한적이다.

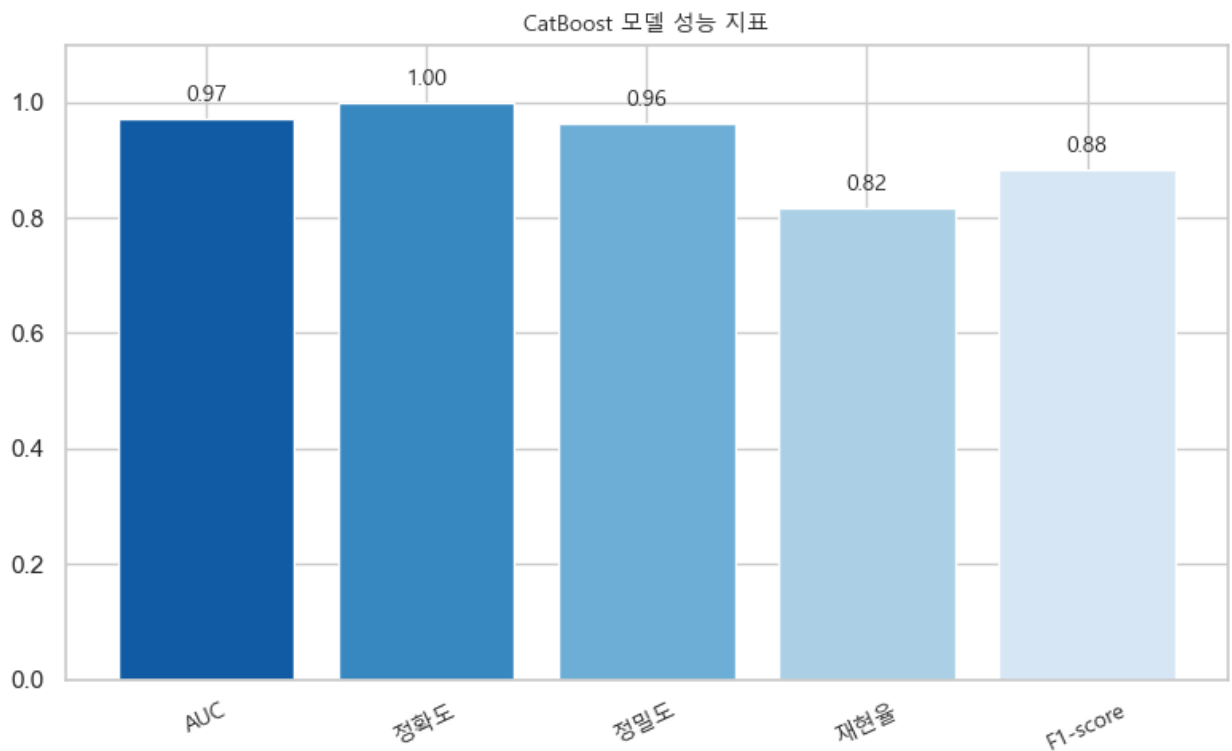


[그림 3.4.5 LVC 모델 지표]

3.4.2.5 CatBoost (CB) 모델 학습

범주형 변수 처리와 불균형 데이터 환경에서 강점을 가진 CatBoost 분류 모델은 학습 효율성과 일반화 성능을 고려해 $\text{depth}=4$, $\text{iterations}=550$, $\text{learning_rate}\approx 0.0178$ 으로 설정되었으며, 규제 강화를 위해 $\text{l2_leaf_reg}\approx 7.55$ 를 적용하였다. 또한 $\text{bagging_temperature}$ 와 random_strength 를 활용하여 의사결정 경로의 다양성을 확보함으로써 과적합을 억제하고 안정적인 학습을 수행할 수 있도록 구성하였다. 성능 평가 결과, AUC 는 0.9707 로 높은 분류 성능을 보였으며, 전체 정확도는 0.9996 으로 매우 높은 수준을 기록하였다. Precision 은 0.9639, Recall 은 0.8163, 그리고 F1-score 는 0.884 로 나타났으며, 이는 모델이 정확한 탐지(Precision)와 놓치지 않는 탐지율(Recall) 간 균형을 성공적으로 확보한 구조임을 의미한다. 특히 F2-score 가 0.8421 로 확인되면서 Recall 가중 환경에서도 안정적인 성능을 유지하는 것으로 나타났다.

종합적으로, 본 CatBoost 모델은 불균형 데이터 환경에서도 높은 분류 정확도와 균형 잡힌 성능을 제공하는 모델로 평가된다.

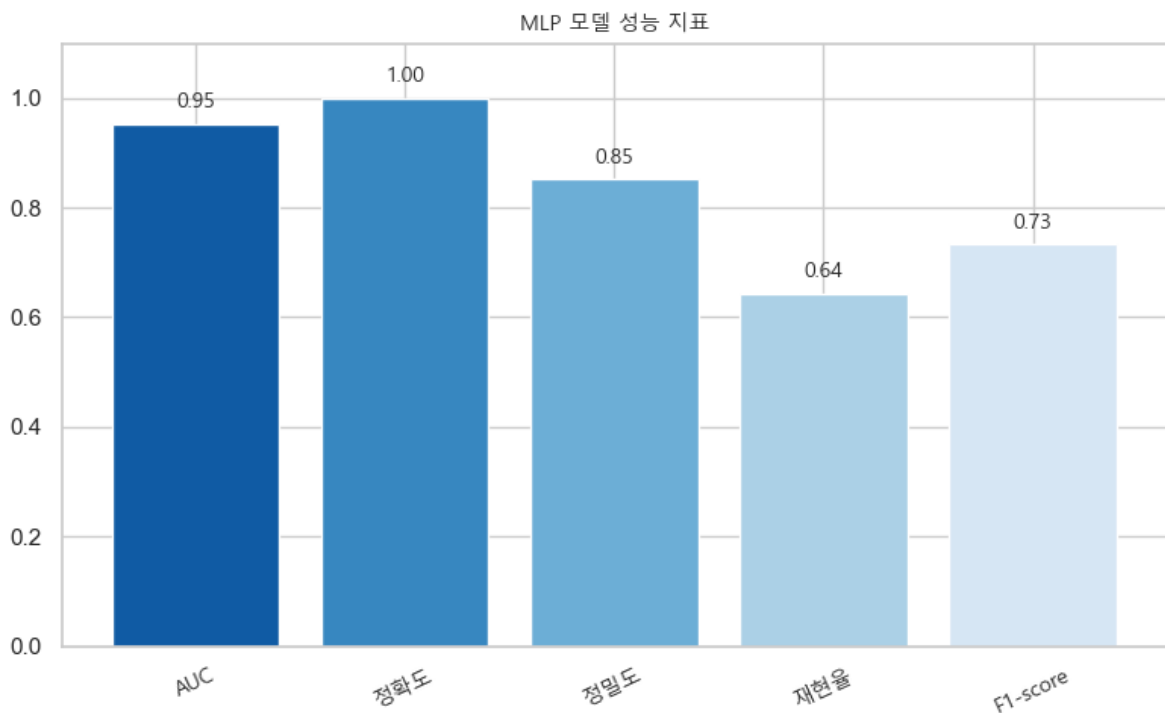


[그림 3.4.6 CatBoost 모델 지표]

3.4.2.6 MLPClassifier 모델 학습

인공신경망 기반 분류 모델인 ****MLP(Multi-Layer Perceptron)**** 모델은 단일 은닉층 구조(hidden_layer_sizes=100)로 구성되었으며, 활성화 함수는 비선형 패턴 학습에 적합한 tanh 를 사용하였다. 학습 과정에서는 solver="adam"을 적용하여 가중치 최적화를 수행하였으며, 과적합 방지 및 수렴 안정성을 위해 early_stopping=True 를 설정하였다. 또한 학습률은 learning_rate="adaptive" 방식으로 설정되어 학습 진행 상황에 따라 자동으로 조정되었으며, 규제 강도(alpha≈0.0047)는 모델 복잡도를 제어하는 방향으로 적용되었다.

성능 평가 결과, AUC 는 0.9528 로 측정되어 클래스 간 구분 성능은 준수한 수준을 보였다. Accuracy 는 0.9992 로 높게 나타났으나, 데이터 불균형 구조를 고려할 때 단독 지표로 해석하기에는 한계가 있다. Precision 은 0.8514, Recall 은 0.6429, F1-score 는 0.7326 으로 나타났으며, 이는 모델이 탐지된 사례 중 올바르게 분류된 비율은 높은 편이지만 일부 이상 거래를 놓치는 경향(False Negative)이 존재함을 의미한다. 즉, 본 모델은 Precision 중심 성향을 보이며 False Positive 를 최소화하는 방향으로 동작하지만, Recall 측면에서는 상대적으로 제한된 탐지 성능을 나타낸다.



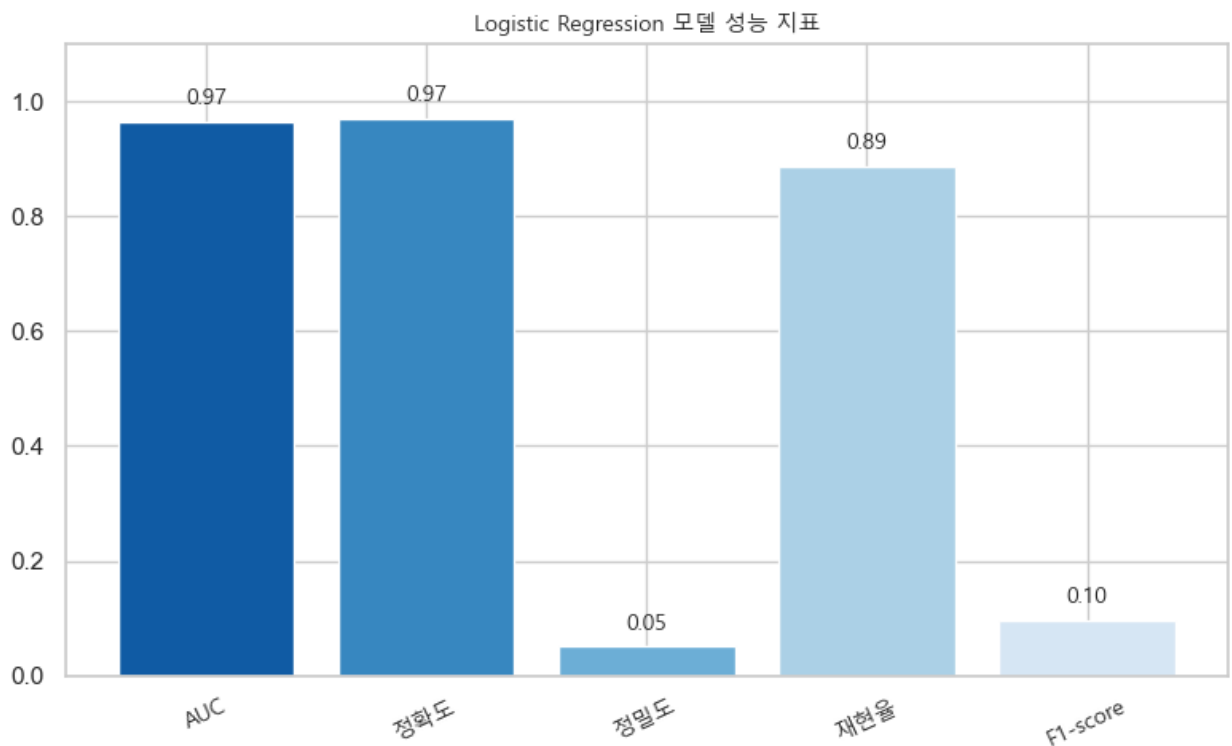
[그림 3.4.7 MLP 모델 지표]

3.4.2.7 LogisticRegression 모델 학습

선형 분류 모델인 Logistic Regression에서는 데이터 불균형 문제를 반영하기 위해 `class_weight="balanced"`를 사용하였으며, 규제 강도(C)는 48.99로 설정되어 비교적 복잡한 결정 경계를 학습할 수 있도록 구성하였다.

성능 평가 결과, ROC-AUC는 0.9657로 클래스 간 구분 능력은 우수한 수준을 보였다. Recall은 0.8878로 높은 탐지 성능을 보였지만, Precision은 0.0504로 매우 낮아 False Positive 비율이 높은 경향을 나타냈다. Accuracy는 0.9710, F1-score는 0.0954로 측정되었으며, Precision-Recall 균형은 다소 부족한 것으로 확인되었다.

종합적으로, 본 Logistic Regression 모델은 이상 거래를 놓치지 않는 방향(Recall 중심)에 적합하다.



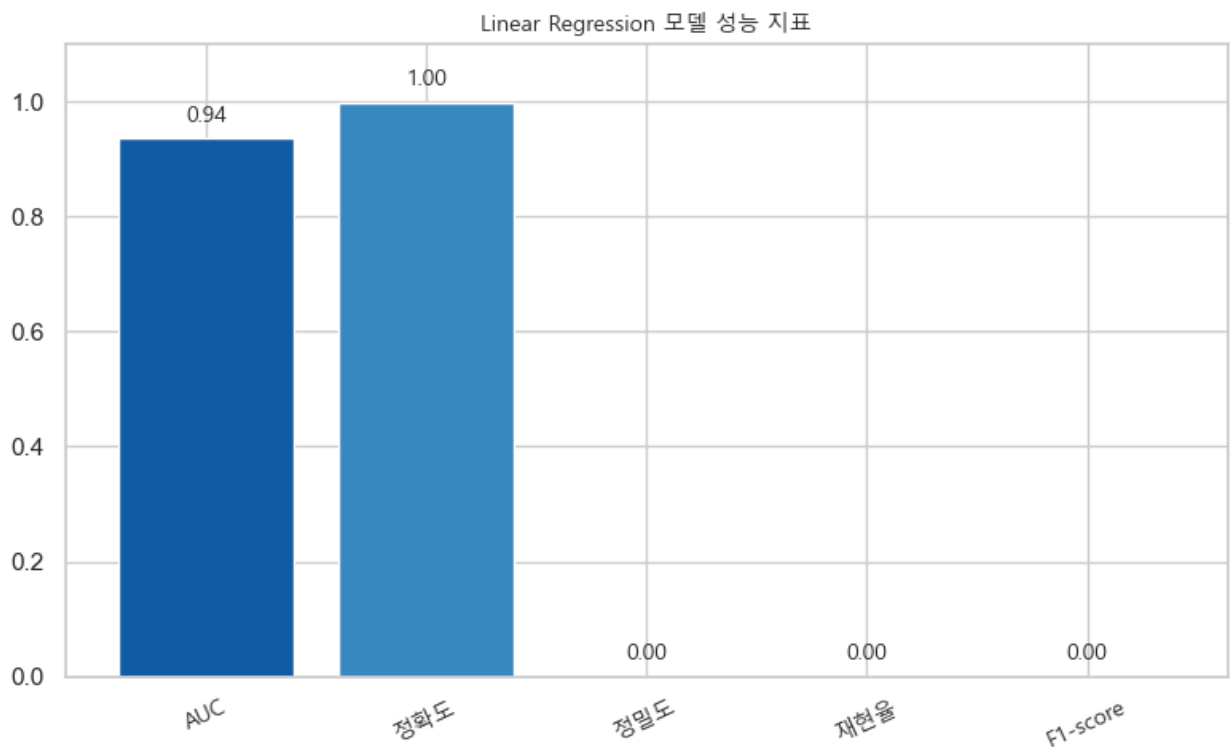
[그림 3.4.8 Logistic Regression 모델 지표]

3.4.2.8 LinearRegression 모델 학습

회귀 기반 접근인 Linear Regression 모델은 `fit_intercept=True` 로 절편을 포함해 학습하도록 설정되었으며, `positive=True` 를 통해 회귀 계수가 음수가 되지 않도록 제한하였다.

성능 결과, **AUC 는 0.9358** 로 비교적 낮은 구분 성능을 보였으며 Accuracy 는 **0.9983** 이었지만, Precision·Recall·F1-score 가 모두 **0.0000** 으로 나타났다. 이는 모델이 소수 클래스(이상 거래)를 전혀 식별하지 못하고 **모든 샘플을 정상으로 예측한 결과**다.

종합적으로 Linear Regression 모델은 본 문제 유형과 데이터 특성에 적합하지 않으며, 이상 탐지 성능 측면에서 활용 가치가 낮은 것으로 평가된다.



[그림 3.4.8 Linear Regression 모델 지표]

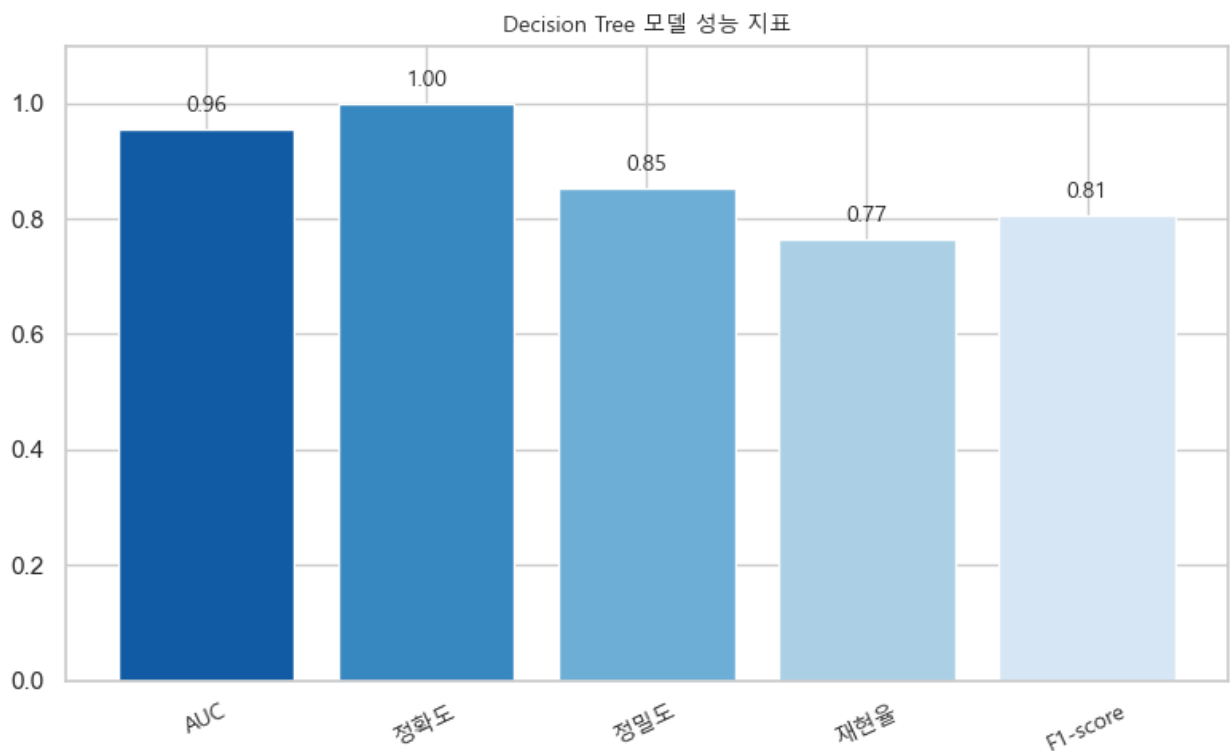
3.4.2.9 DecitionTree 모델 학습

단일 트리 기반 분류 모델인 Decision Tree 모델은 과적합을 방지하기 위해 `max_depth=4`, `min_samples_split=3`, `min_samples_leaf=8` 로 제한하여 단순화된 구조로 학습되었으며, 분기 기준은 `entropy` 를 적용하여 정보 이득 기반 분류를 수행하였다.

성능 평가 결과, ROC-AUC 는 0.9552 로 나타났으며, Precision 0.8523, Recall 0.7653, F1-score 0.8065 로 비교적 균형 잡힌 탐지 성능을 보였다. Accuracy 는 0.9994 로 높게 나타났으나, 데이터 불균형 특성을 고려하면 단독 해석보다는 AUC 와 F1-score 중심으로 평가하는 것이 적절하다.

종합적으로 Decision Tree 모델은 해석 가능성과 간단한 구조 대비 안정적인 성능을 제공한 모델로 평가되며, 특히 Precision 과 Recall 간 균형 측면에서 baseline 모델로 활용 가능한 수준의 결과를 보였다.

Decision Tree 모델은 높은 정밀도와 안정적인 F1-score 를 기반으로 비교적 균형 잡힌 분류 성능을 보였다. 재현율도 0.7653 수준으로 사기 거래 탐지가 일정 부분 확보된 결과이며, 깊이가 제한된 모델 구조(Depth 4)로 인해 과적합 억제 효과가 반영된 것으로 해석된다.



[그림 3.4.8 Decision Tree 모델 지표]

3.4.3 하이퍼파라미터 튜닝 (HyperOpt 적용)

3.4.3.1 HpyerOpt 활용 최적 파라미터 탐색 및 도출

모델 성능 향상을 위해 HyperOpt 기반의 Bayesian Optimization 방식을 활용하여 하이퍼파라미터 탐색을 수행하였다. 탐색 과정에서 TPE(Tree-structured Parzen Estimator) 알고리즘을 적용하여 무작위 탐색 대비 효율적인 탐색이 가능하도록 구성하였으며, 각 후보 파라미터는 교차 검증 성능(ROC-AUC)을 기준으로 평가하였다. 또한 탐색 과정에서는 모델 생성 오류 처리, 파라미터 타입 변환 등을 포함하여 다양한 모델 유형(SVM, Tree Model, Boosting Model 등)에 대해 일관된 최적화가 가능하도록 설계하였다. 총 탐색 횟수(max_evals) 동안 가장 높은 점수를 기록한 조합을 최종 최적 파라미터로 선정하였으며, 탐색 결과는 재현성과 활용성을 위해 Trials 객체 및 JSON 파일 형태로 저장하였다.

```
# 탐색 범위를 설정하고 다양한 모델 유형에 대해 적용

search_space = {
    'C': hp.loguniform('C', -5, 5),
    'penalty': hp.choice('penalty', ['l2']),
    'max_iter': hp.quniform('max_iter', 100, 500, 50),
    'class_weight': hp.choice('class_weight', ['balanced'])
}

# ROC-AUC 기반 교차 검증으로 평가

def objective(params):
    model = model_class(**params)

    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='roc_auc')

    return {'loss': -scores.mean(), 'status': STATUS_OK}

# 총 탐색 횟수 동안 최적 값을 탐색

trials = Trials()

best_params = fmin(
```

```

fn=objective,

space=search_space,

algo=tpe.suggest,

max_evals=100,

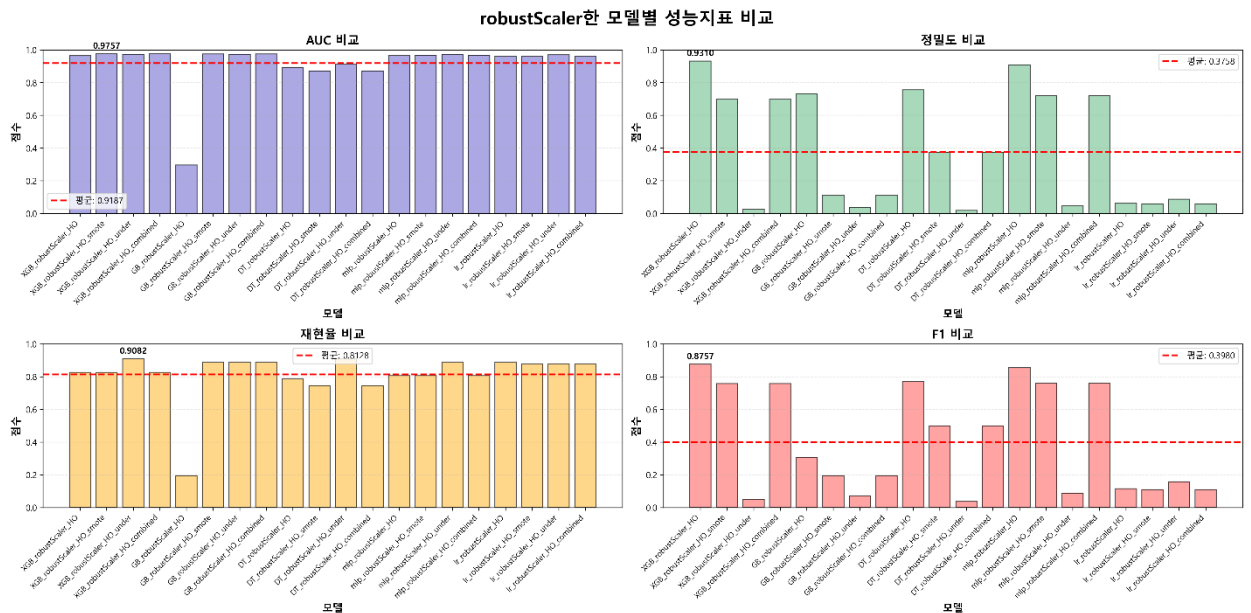
trials=trials

)

```

3.4.3.2 다양한 스케일링/샘플링 조합에 대한 성능 비교

RobustScaler 모델별 성능



[그림 3.4.9 RobustScaler 모델 지표]

데이터의 이상치(Outlier) 영향을 최소화하기 위해 RobustScaler를 적용한 뒤, 모델(XGBoost, GradientBoosting, DecisionTree, MLP, Logistic Regression)과 샘플링 방식(None, Oversampling, Undersampling, Combined Sampling) 조합에 따른 성능 변화를 비교하였다.

1) AUC 기준

- A. 대부분의 모델 조합에서 0.9 이상의 성능을 유지하여 모델 분류 경계 설정에는 비교적 안정적인 성능을 보였다.

- B. 일부조합 (예: DT_robustscale_HO_none)에서는 AUC 가 급격히 감소하여 스케일링과 샘플링 방식이 모델에 따라 상호작용 효과를 가진다.

2) Precision(정밀도) 기준

- A. 정밀도는 모델 간 편차가 매우 컸으며, 일부 모델은 높은 정밀도를 기록했지만 다수 조합은 0.1 이하의 낮은 수치를 보였다. 이는 RobustScaler 적용이 정밀도 향상에 크게 기여하지 못했음을 의미한다

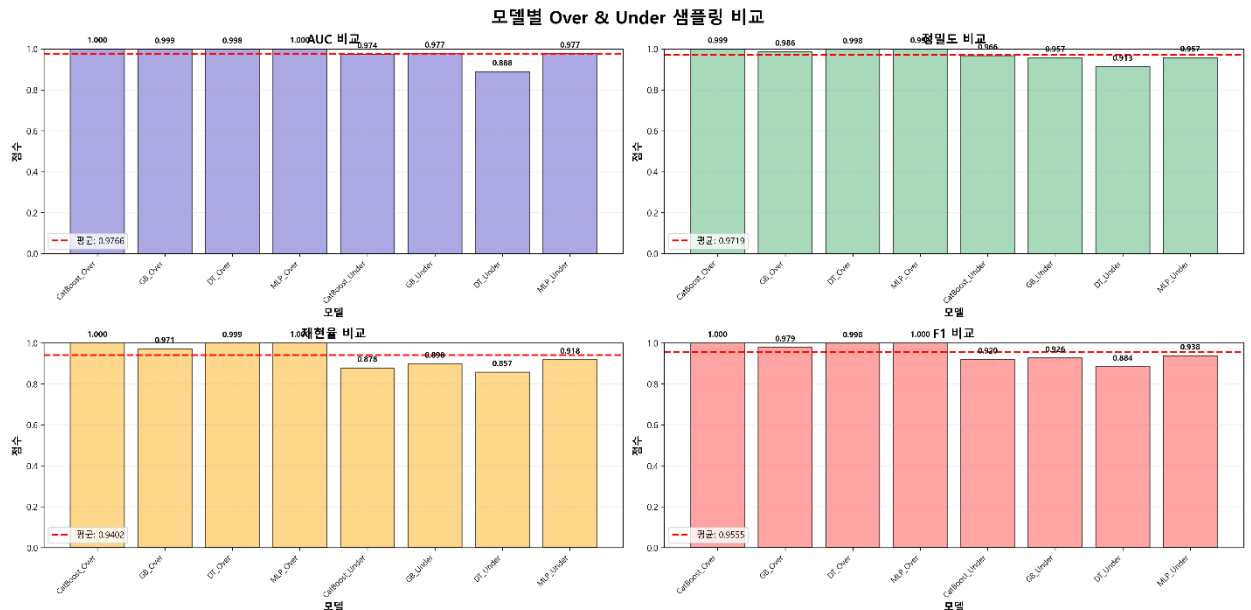
3) Recall(재현율) 기준

- A. Recall 은 Precision 대비 상대적으로 높은 수치를 유지하였으며, 일부 조합에서는 0.9 수준에 도달하였다. 이는 RobustScaler 가 이상치의 영향을 줄여 민감도를 향상시키는 방향으로 작용할 수 있음을 시사한다.

4) F1-score 기준 종합 해석

- A. F1-score 는 정밀도와 재현율의 균형을 보여주는 지표로, 전체 실험 중 XGBoost + Oversampling 조합이 가장 우수한 성능($F1 \approx 0.87$) 을 기록했다. 반면 다른 조합들에서는 F1 점수가 급격히 감소하여, RobustScaler 자체보다 Sampling 전략 및 모델 구조가 성능 결정에 더 중요한 요인임을 확인할 수 있었다.

Over & Under 샘플링



[그림 3.4.10 Over&Under sampling 지표]

1) Oversampling

- A. 대부분의 모델에서 AUC, Precision, Recall, F1-score 가 0.98~1.0 에 근접
특히 MLP 와 Cat Boost 는 거의 완벽한 모델 성능을 보인다, 이와 같은
결과는 데이터 분포가 균형화 되면서 모델이 학습 데이터에 과도하게
적응(overfitting) 했을 가능성이 높음을 의미한다.

2) Under sampling

- A. 전반적으로 성능은 Oversampling 대비 낮아졌으나, 과적합 현상이
완화되며 보다 현실적인 평가 결과를 제공 MLP 와 GradientBoosting 은
샘플링 변화에도 비교적 성능 일관성을 유지 반면 DecisionTree 모델은
Undersampling 환경에서 성능 하락폭이 크게 나타난다.

3) 결론

- A. Oversampling 방식은 모든 모델에서 성능이 극단적으로 향상되는 반면,
Undersampling 방식은 보다 현실적인 모델 성능 평가를 제공하였다. 특히

MLP 모델은 두 샘플링 전략에서 모두 우수한 성능을 유지하여 최종 후보 모델로 선정할 수 있다.

3.4.4 최종 앙상블 모델 구성 및 학습

3.4.4.1 소프트 보팅 앙상블 전략 정의

소프트 보팅 앙상블(Soft Voting Ensemble)은 여러 개의 분류 모델이 예측한 클래스별 확률값을 평균하거나 가중합하여 최종 예측 결과를 결정하는 방식이다. 단순히 다수결로 투표하는 하드 보팅과 달리 모델이 예측한 확률 정보를 활용하여 보다 안정적이고 균형 잡힌 예측을 수행한다.

3.4.4.2 개별모델/스태킹 결과 기반 최종 모델 구성 (stack1~5 기반 투표)

- 목적

단일 모델 기반 학습 결과 모델 간 성능 편차가 존재하였으며, 특정 모델은 특정 데이터 샘플링 조건에서 강점을 보였다. 이를 기반으로 서로 다른 학습 특성을 가진 모델을 조합한 Stacking Ensemble 전략을 추가적으로 수행하여 모델 일반화 성능을 향상시키고자 하였다.

Stack	포함 모델	설계 의도
Stack 1	CatBoost + XGBoost + LightGBM	Gradient Boosting 계열 최적 조합
Stack 2	LightGBM + RandomForest + MLP	트리 기반 + 신경망 혼합 구조
Stack 3	CatBoost + GradientBoosting + SVM(RBF)	비선형 모델 조합
Stack 4	XGBoost + Linear SVM + LogisticRegression	선형 + 트리 기반 조합
Stack 5	RandomForest + SGD + MLP	전통 ML + Neural Hybrid

스태킹 구조는 모델 간 예측 방식(트리 기반/선형/신경망)이 상호 보완되도록 설계하였다.

각 스태킹 모델은 동일한 데이터셋에 대해 학습하였으며, 메타모델(final estimator)은 성능이 안정적이고 interpretability 가 우수한 Logistic Regression 으로 설정하였다.

모델명	AUC	정확도	정밀도	재현율	F1	F2
Stack 1	0.9722	0.9877	0.1108	0.8776	0.1968	0.3682
Stack 2	0.9719	0.9957	0.2669	0.8469	0.4059	0.5903
Stack 3	0.9702	0.9956	0.2642	0.8571	0.4038	0.5915 (↑ 최고)
Stack 4	0.9734(↑ 최고)	0.988	0.1138	0.8776	0.2014	0.3746
Stack 5	0.97	0.9905	0.1398	0.8776	0.2412	0.427

표의 결과에서 볼 수 있듯이, 모든 스택킹 모델은 AUC 기준 약 0.97 이상의 높은 분류 성능을 보였으나, 정밀도와 재현율의 균형 측면에서는 모델별 차이가 존재하였다. 특히 Stack 2와 Stack 3은 상대적으로 높은 F1 및 F2 스코어를 기록하며 가장 균형 잡힌 성능을 나타냈으며, 이는 트리 기반 모델과 신경망/비선형 모델의 조합이 예측 안정성 향상에 기여했음을 의미한다. 반면 Stack 1과 Stack 4는 높은 재현율 대비 정밀도 저하가 두드러졌는데, 이는 해당 조합이 False Positive 증가로 이어졌음을 보여준다.

3.4.4.3 가중치 부여를 통한 앙상블 최적화 (weights=[3,2,2,1,1])

본 단계에서는 앞서 생성한 5개의 스택킹 모델을 Soft Voting 방식으로 결합하고, 각 모델의 성능 기여도와 역할을 반영하여 가중치를 차등 적용하였다. 해당 가중치(3, 2, 2, 1, 1)는 모델별 Validation 성능(F1/F2 지표), 예측 편향(Precision vs Recall), 구조적 다양성(트리 기반·선형·신경망 모델 등)을 고려해 결정하였다. 이를 통해 단순 평균 Voting 대비 성능 기반 의사결합(weighted decision aggregation)이 가능하도록 설계하였다.

Soft Voting을 적용한 이유는 모델이 출력하는 예측 확률값(predicted probability)을 기반으로 결과를 결합함으로써, 개별 모델의 약점을 상쇄하고 일반화 성능을 개선할 수 있기 때문이다.

아래는 Soft Voting 적용 후 최종 모델의 성능 결과이다.

평가 지표	성능
AUC	0.9719
Precision	0.2727
Recall	0.8571
F1 Score	0.4138

최종 결과는 개별 스택킹 모델 대비 Precision 과 Recall 간 균형이 개선된 형태를 보였으며, 특히 Fraud Detection 목적상 중요한 재현율(Recall)을 유지하면서도 Precision 이 향상된 점에서 앙상블 전략 적용의 효과가 확인되었다.

3.5 모델 평가 및 해석

본 절에서는 신용카드 사기 거래 탐지 모델의 성능을 다각도로 평가하고, 최종 선정된 모델의 특성과 한계를 해석한다. 본 프로젝트는 극심한 클래스 불균형 문제를 해결하기 위해 Full-data, Under-sampling, Over-sampling(SMOTE 기반) 등 다양한 데이터 처리 전략을 적용하였으며, 팀원 5 명이 약 200 여 회의 실험을 수행하면서 모델 성능을 최대화하는 데 집중하였다. 특히 오버샘플링 기반 모델들에 대해 F1 Score 기준 상위 10 개 모델을 선별하여 정량·정성 평가를 진행하였다.

3.5.1 최종 평가 지표 분석

모델 성능 평가는 Precision(정밀도), Recall(재현율), F1 Score, ROC-AUC 를 중심으로 수행하였다. 전체 실험 모델(약 200 회 실험)에 대한 성능 지표 분포와 통계 요약은 표 3.5.1-1 및 그림 3.5.1-1~3.5.1-4 에 정리하였다.

표 3.5.1-1 주요 지표 통계 요약

지표	최고값	최저값	평균	표준편차
AUC	1.0000	0.9774	0.9963	0.0070
정밀도(Precision)	0.9994	0.9574	0.9902	0.0133
재현율(Recall)	1.0000	0.9184	0.9792	0.0301
F1	0.9997	0.9375	0.9846	0.0217

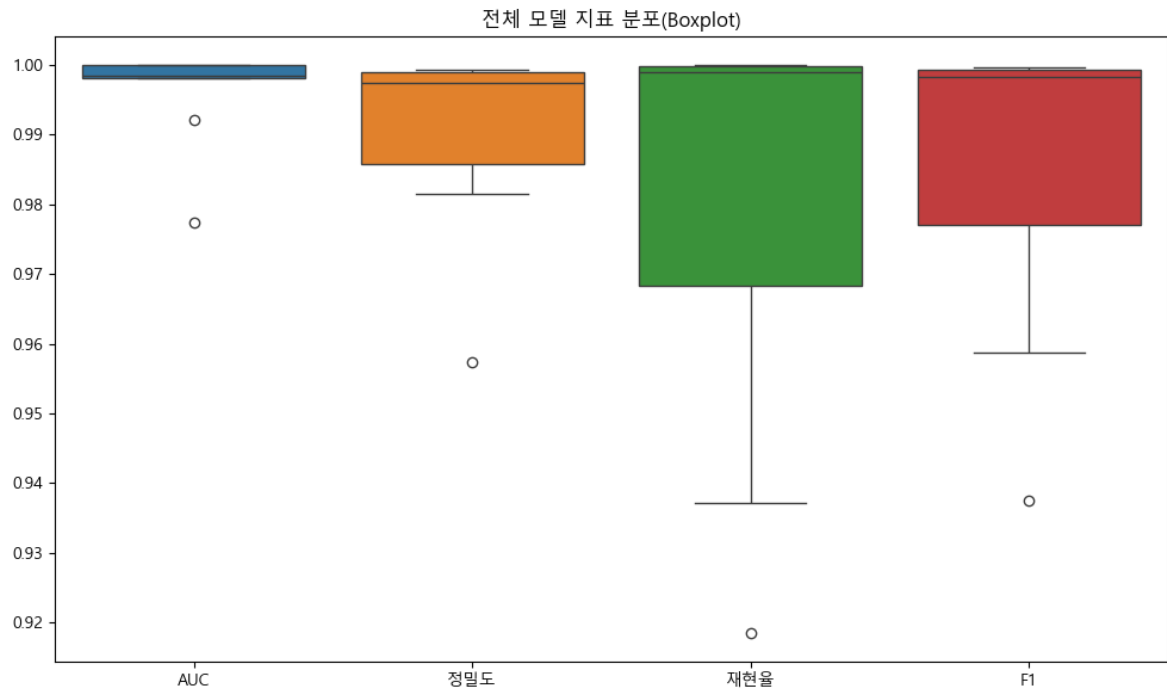


그림 3.5.1-1 전체 모델 지표 분포(Boxplot)

그림 3.5.1-1 에서 AUC, 정밀도, 재현율, F1 모두 전반적으로 0.98 이상 구간에 밀집되어 있으며, 특히 AUC 는 대부분 0.995 이상으로 매우 안정적인 분류 성능을 보인다.

재현율의 분산이 상대적으로 크다는 점은 모델에 따라 사기 거래 탐지 민감도가 다르게 나타난다는 것을 의미하며, 본 프로젝트에서는 재현율과 F1 을 핵심 지표로 두고 추가 분석을 수행하였다.

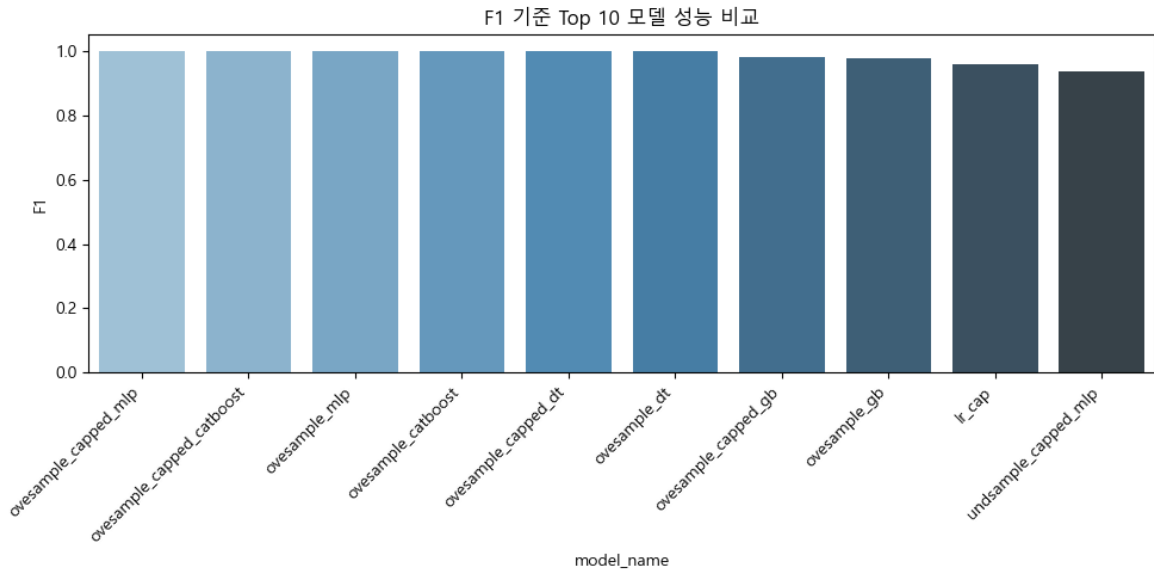


그림 3.5.1-2 F1 기준 Top 10 모델 성능 비교

그림 3.5.1-2 는 F1 Score 기준 상위 10 개 모델의 성능 비교 결과이다.

oversample_capped_mlp, oversample_capped_catboost, oversample_mlp, oversample_catboost 등 오버샘플링 기반 모델들이 최상위권을 형성하였고, F1 Score 는 모두 0.99 내외로 동급의 최고 성능을 기록하였다. 반면 언더샘플링 기반 모델은 상대적으로 낮은 성능을 보였는데, 이는 언더샘플링 과정에서 정상 거래 정보가 과도하게 제거되며, 사기 거래와의 분리 경계 학습에 필요한 패턴 정보가 손실되었기 때문으로 해석된다.

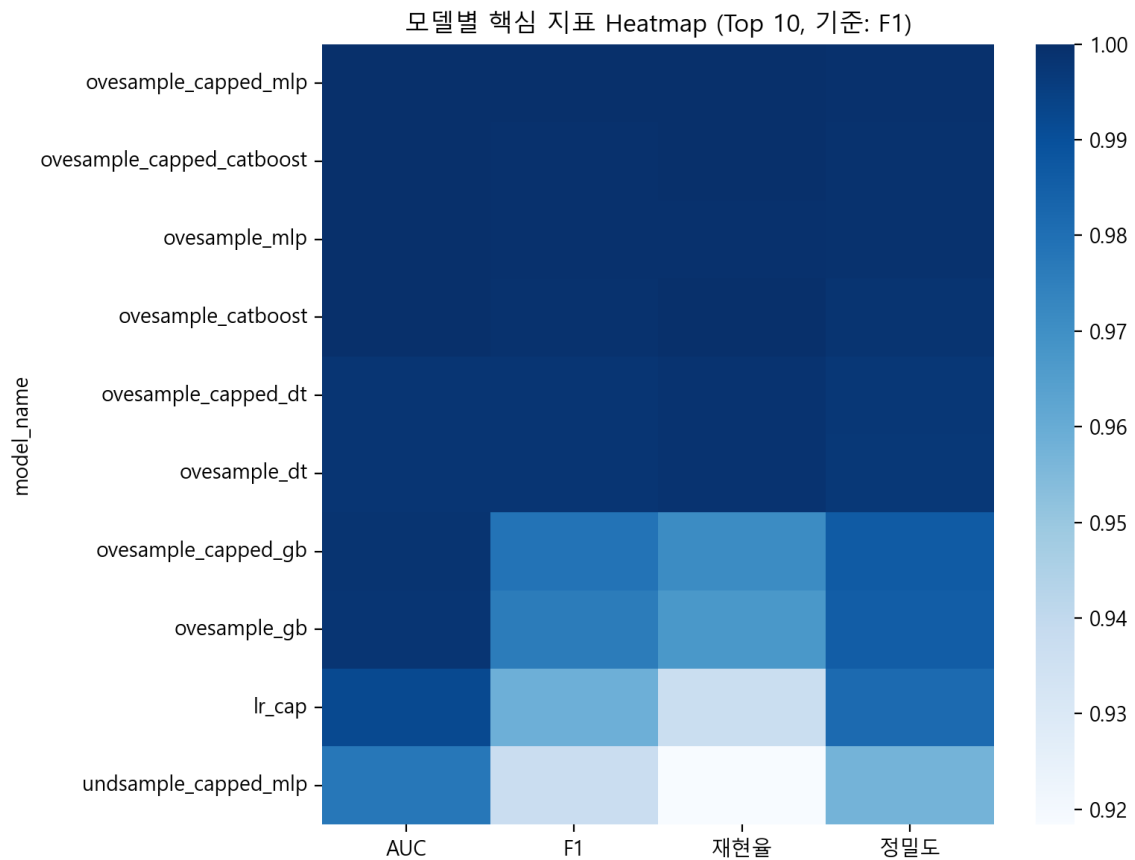


그림 3.5.1-3 Top 10 모델 핵심 지표 Heatmap

그림 3.5.1-3의 Heatmap은 F1 기준 Top 10 모델에 대해 AUC, F1, 재현율, 정밀도를 동시에 비교한 결과이다. 상위 5개 모델은 네 지표 모두에서 고르게 우수하며, 일부 모델은 재현율 또는 정밀도에서 미세한 편차가 관찰된다. 즉, '최고 성능' 모델을 단일 지표로 결정하기보다는 운영 목적(오탐 허용 수준, 미탐 비용)을 함께 고려해야 한다.

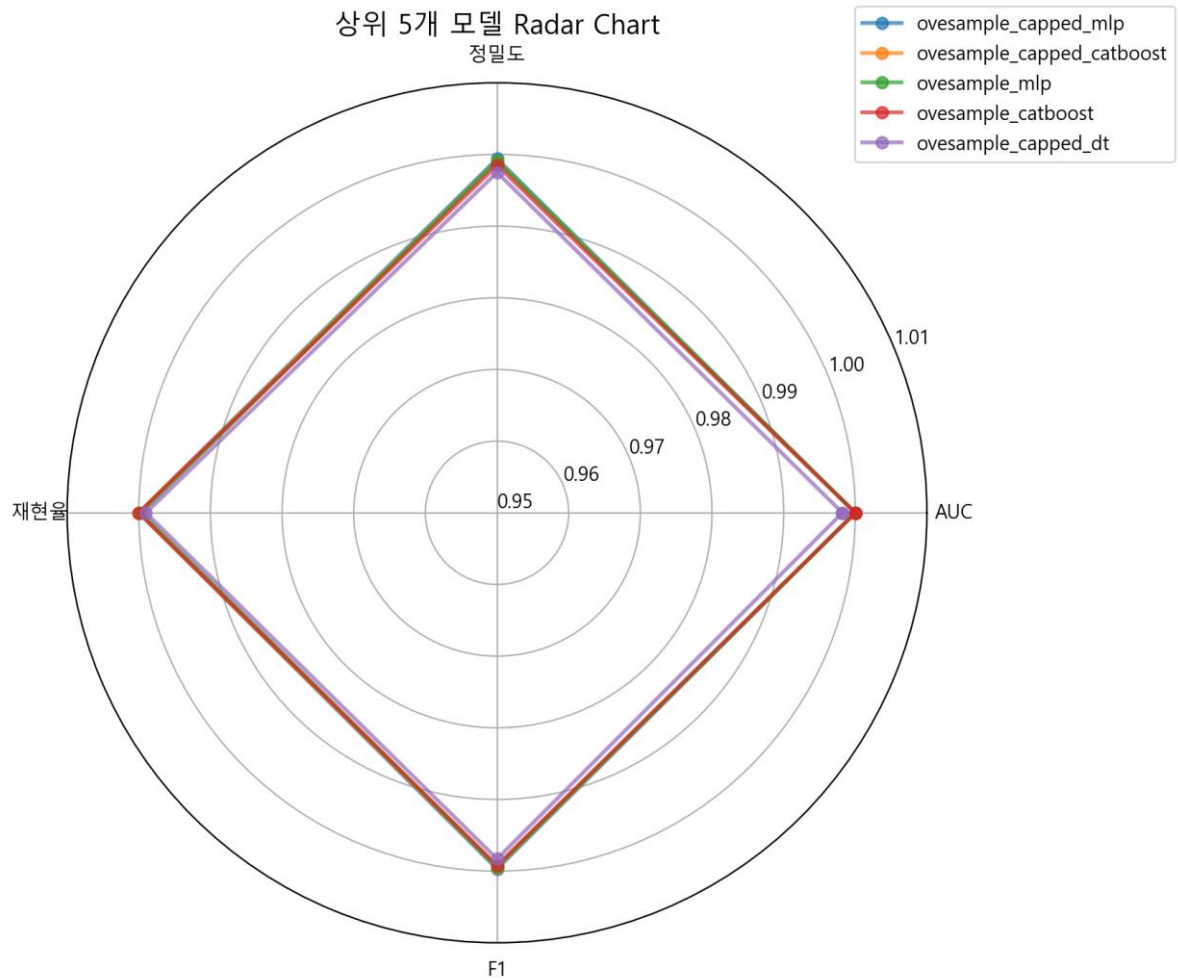


그림 3.5.1-4 상위 5 개 모델 Radar Chart

그림 3.5.1-4 Radar Chart 는 상위 5 개 모델의 지표 균형을 시각적으로 표현한 것이다. 각 모델의 선이 거의 겹칠 정도로 4 개 지표가 모두 0.98~1.0 구간에 집중되어 있어 상위 모델 간 성능 차이가 매우 미미함을 보여준다. 그중 oversample_capped_mlp 는 지표의 변동폭이 작고 전반적으로 높은 값을 유지하여, 균형형 모델로 판단된다.

3.5.1.1 지표 간 비교 요약

전반적으로 재현율이 정밀도보다 약간 더 높은 경향을 보여, 사기 거래를 놓치지 않도록 모델이 조정되었음을 확인하였다. F1 Score 와 AUC 는 상위 모델 대부분이 0.98

이상으로 매우 우수하며, 언더 샘플링 전략을 사용한 모델은 데이터 손실로 인해 상대적으로 낮은 재현율과 F1 Score 를 기록하였다. 이러한 결과를 바탕으로, 이후 분석에서는 오버샘플링 기반 상위 모델들을 중심으로 최종 모델을 선정하였다.

3.5.1.2 최종 선택 모델: MLP Oversampling 모델(oversample_capped_mlp)

최종 선정된 모델은 oversample_capped_mlp 이다. 이 모델은 F1 Score 0.9997, AUC 1.0000, 정밀도 0.9994, 재현율 1.0000 을 기록하여 전체 모델 중 가장 우수한 성능을 보였다. 재현율 1.0000 은 테스트 데이터 기준으로 사기 거래를 단 한 건도 놓치지 않았음을 의미하며, 동시에 정밀도 역시 0.999 이상으로 유지되어 정상 거래에 대한 오탐도 매우 낮은 수준으로 억제되었다. SMOTE 기반 오버샘플링과 상한값(Capping) 전처리, MLP 구조의 조합이 불균형 데이터에서의 분류 경계를 효과적으로 학습한 결과로 해석된다.

3.5.2 Feature Importance 분석

최종 선정된 MLP 모델은 결정트리 계열 모델처럼 명시적인 분기 규칙을 제공하지 않기 때문에, 본 프로젝트에서는 Permutation Importance 및 SHAP 기반 분석을 통해 주요 특성의 기여도를 해석하였다. 데이터 특성상 V1~V28 은 PCA 변환된 특성으로 원천 변수의 직접적인 의미 해석은 제한적이지만, 사기 거래와 정상 거래를 구분하는 '잠재 패턴'의 방향성과 상호작용을 파악하는 데 유효하다.

Permutation Importance(그림 3.5.2-1)와 SHAP 기반 분석(그림 3.5.2-2~3.5.2-3)은 공통적으로 V4, V3, V10 이 모델 예측에 가장 큰 영향을 미치는 상위 특성임을 보여준다. 이는 해당 잠재 변수들이 사기 거래에서 반복적으로 관측되는 비정상 패턴을 강하게 반영하고 있음을 시사한다.

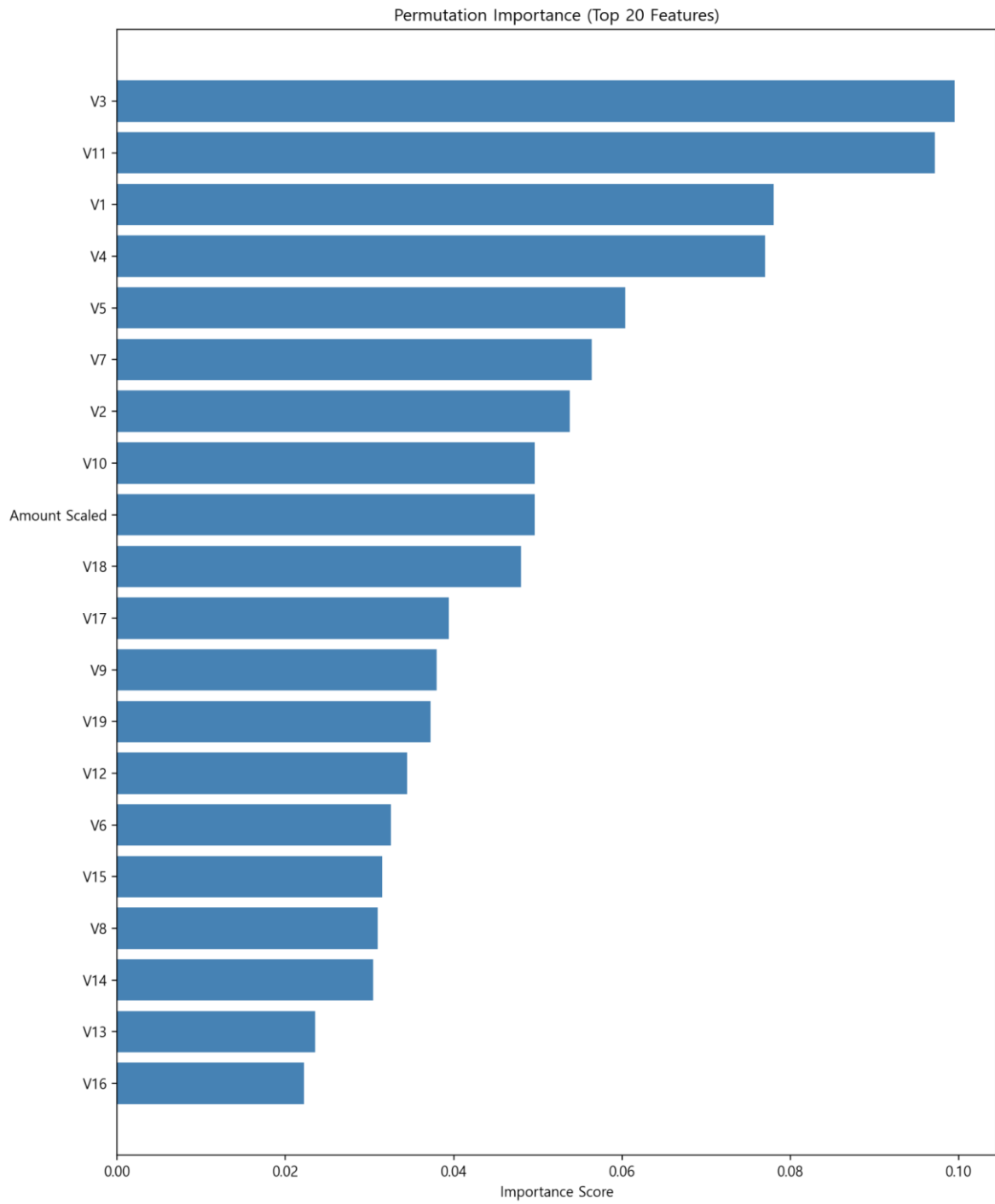


그림 3.5.2-1 Permutation Importance (Top 20 Features)

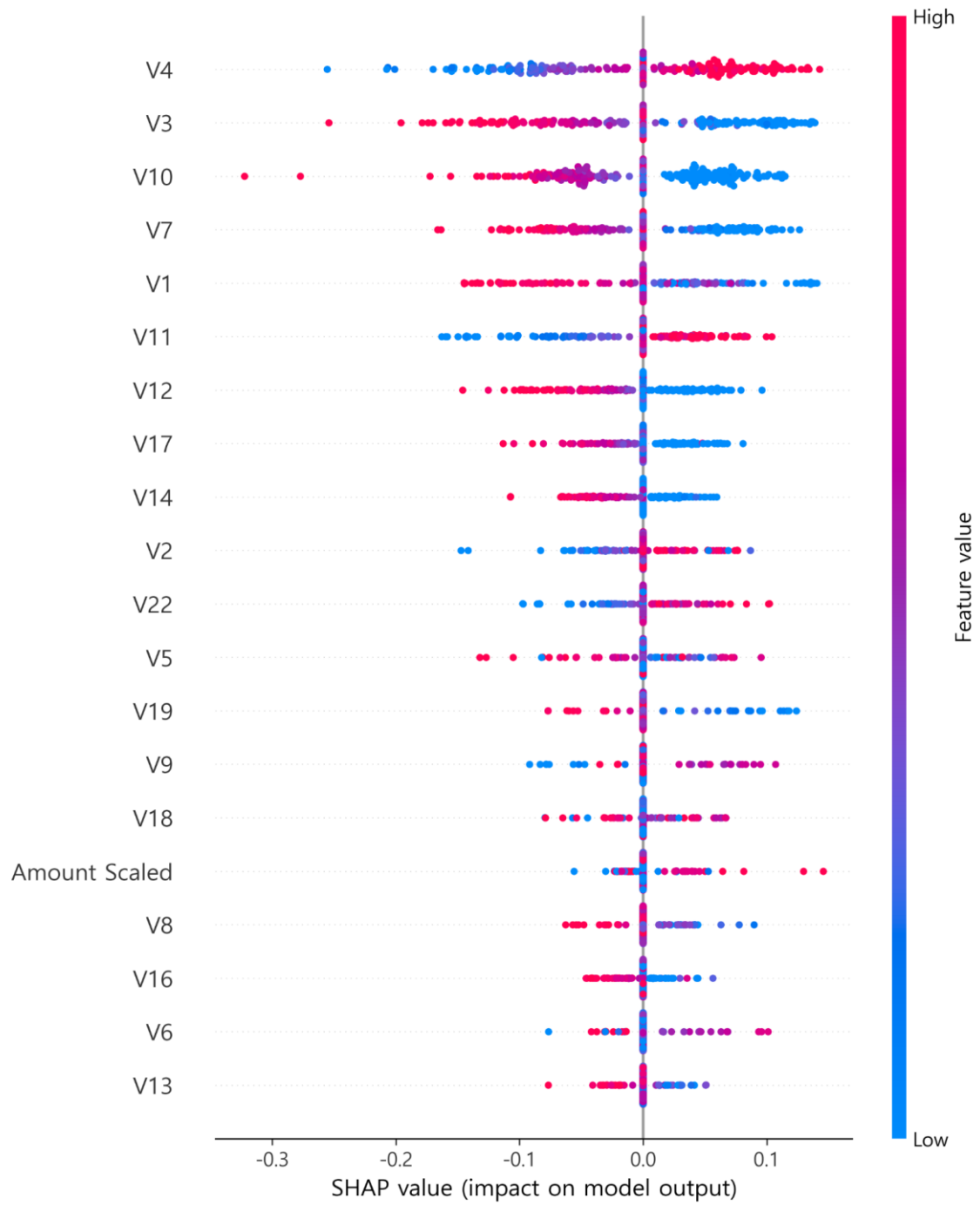


그림 3.5.2-2 SHAP Summary Plot

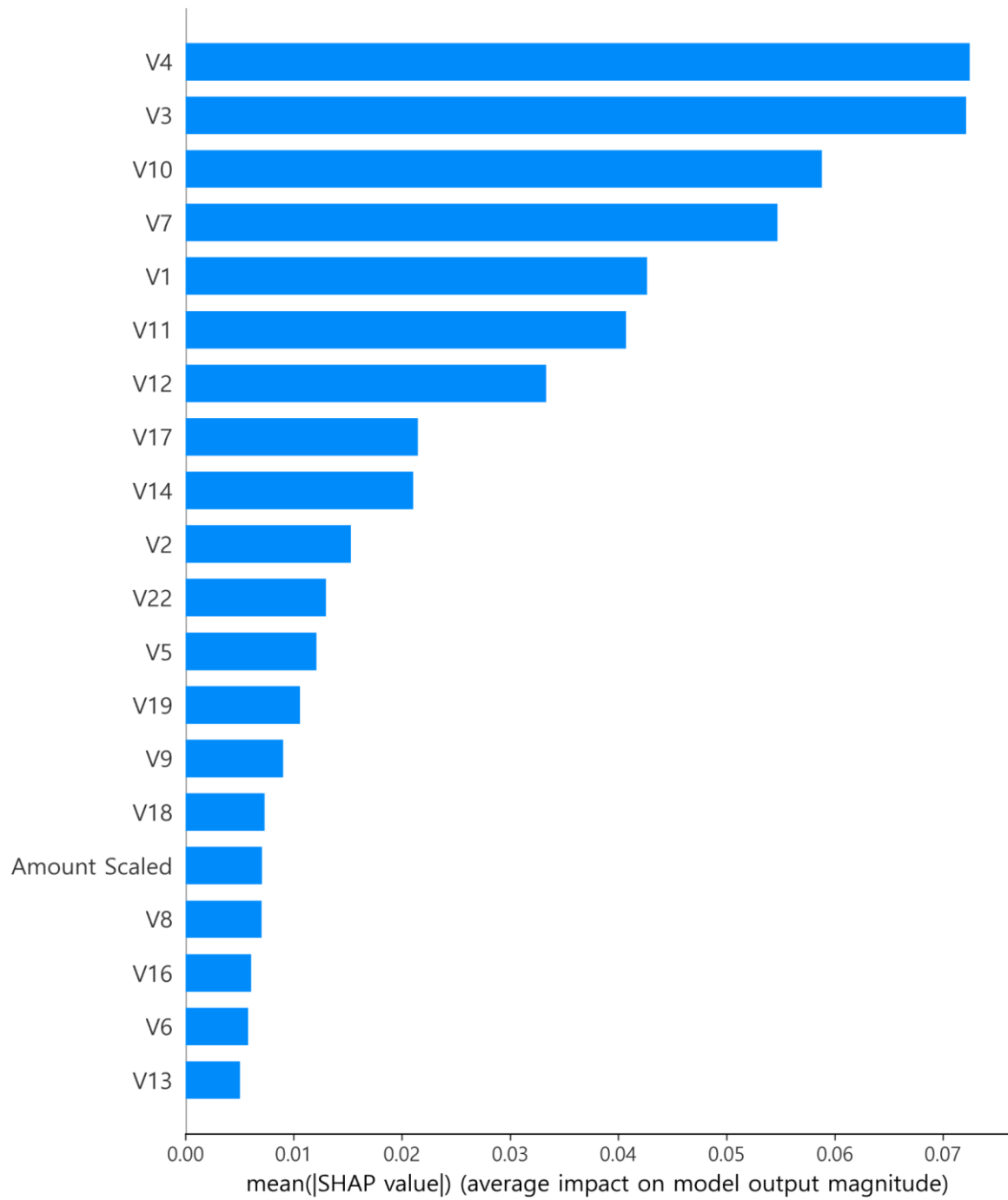


그림 3.5.2-3 SHAP Bar Plot (mean |SHAP| Top 20)

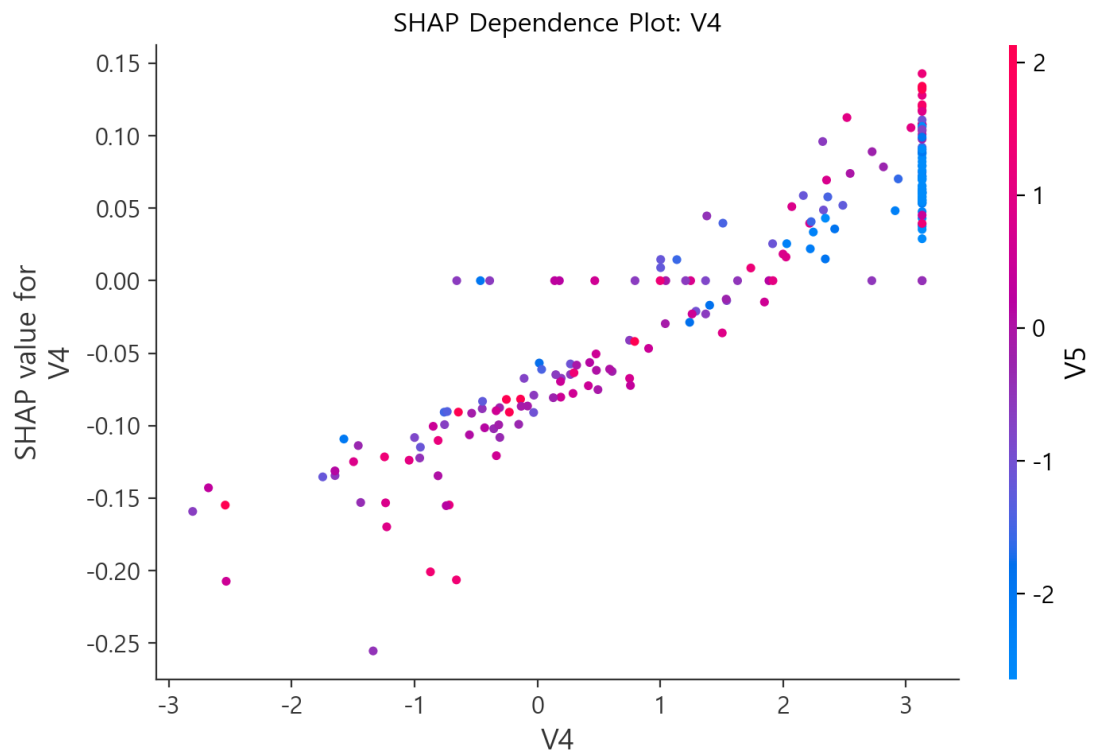


그림 3.5.2-4 SHAP Dependence Plot: V4

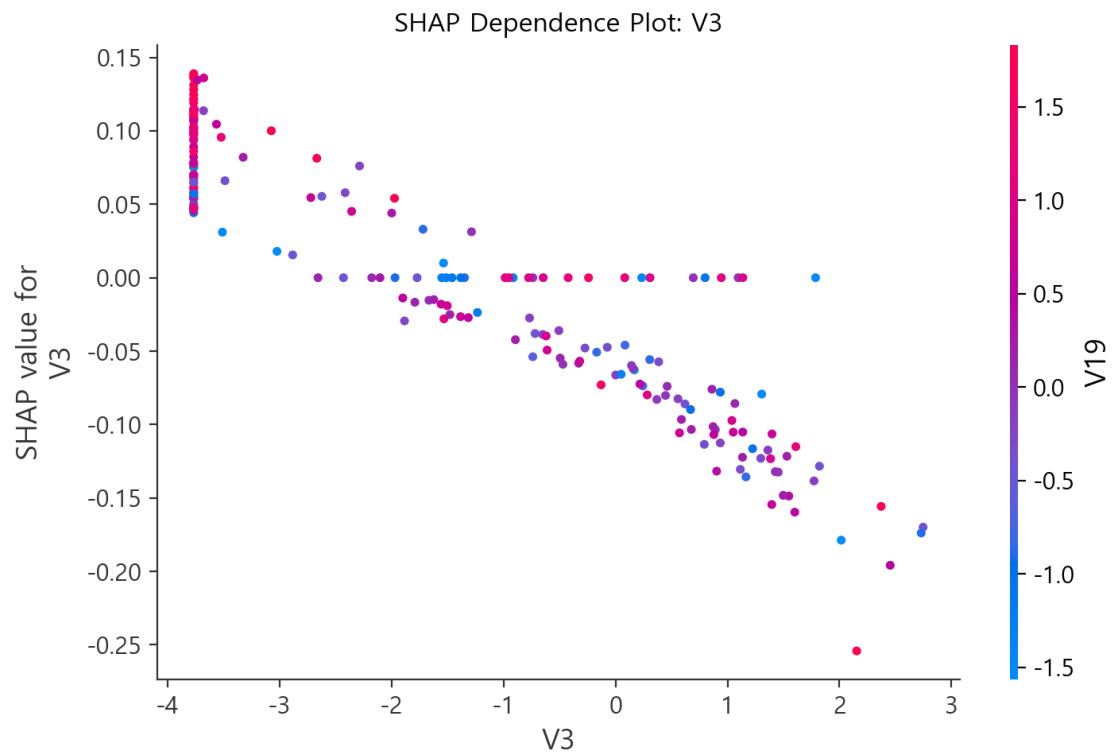


그림 3.5.2-5 SHAP Dependence Plot: V3

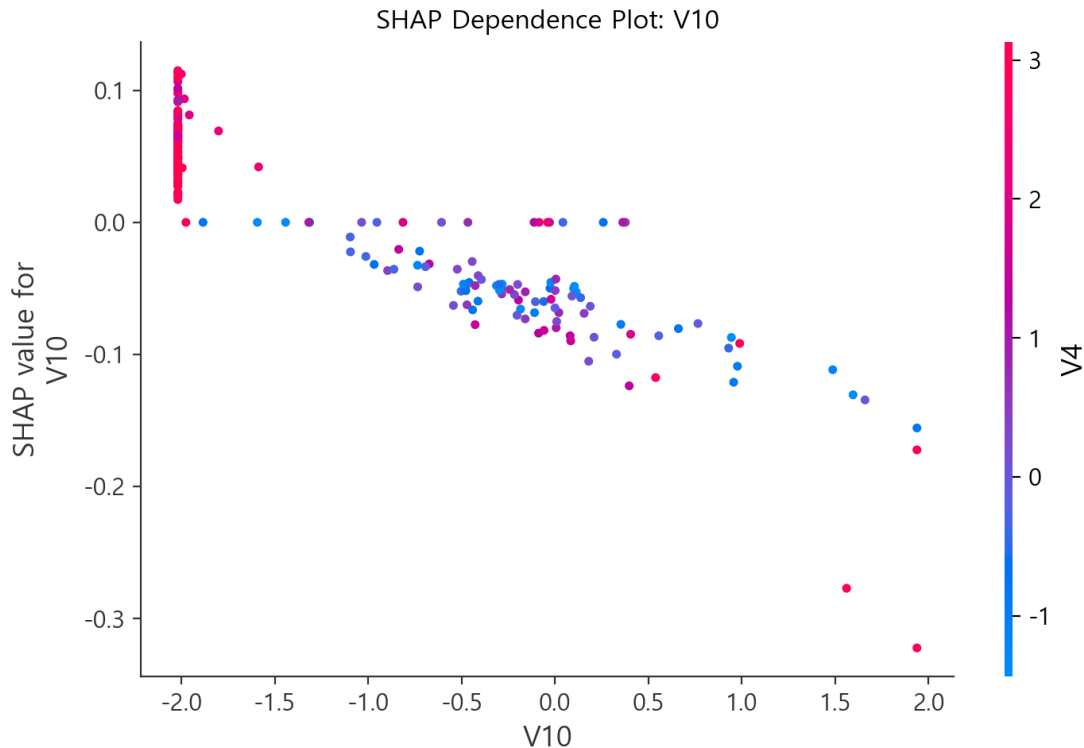


그림 3.5.2-6 SHAP Dependence Plot: V10

SHAP Dependence Plot(그림 3.5.2-4~3.5.2-6)을 통해 상위 특성의 값 변화에 따라 예측 기여도가 단조적으로 증가/감소하는 구간이 관찰되며, 일부 구간에서는 다른 특성(색상으로 표시된 보조 변수)과의 상호작용에 의해 기여도가 달라질 수 있음을 확인하였다. 따라서 운영 적용 시에는 단일 특성의 임계값 규칙보다는, 다변량 패턴을 포착하는 현 모델의 강점을 유지하되 후속 설명(설명가능 AI 리포트)을 함께 제공하는 방식이 적합하다.

향후 설명 가능성을 높이기 위해서는 (1) V 특성의 원천 변수 재구성 가능성 검토, (2) 모델의 국소 설명(Local Explanation) 제공(예: 개별 거래의 SHAP Waterfall), (3) 고위험 구간에 대한 규칙 기반 보조 룰(2 차 심사) 설계 등이 요구된다.

이러한 결과는 본 모델이 단일 변수 기준 규칙이 아닌, 다수 특성의 조합적 패턴을 통해 사기 거래를 식별하고 있음을 의미한다.

3.5.3 예측 결과 시각화 및 오차 분석

예측 결과에 대한 시각화와 오차 분석은 모델의 실무 적용 가능성을 평가하는 데 중요하다. 본 절에서는 임계값(Threshold) 변화에 따른 지표 변화를 중심으로, 운영 환경에서의 의사결정 기준을 검토하였다.

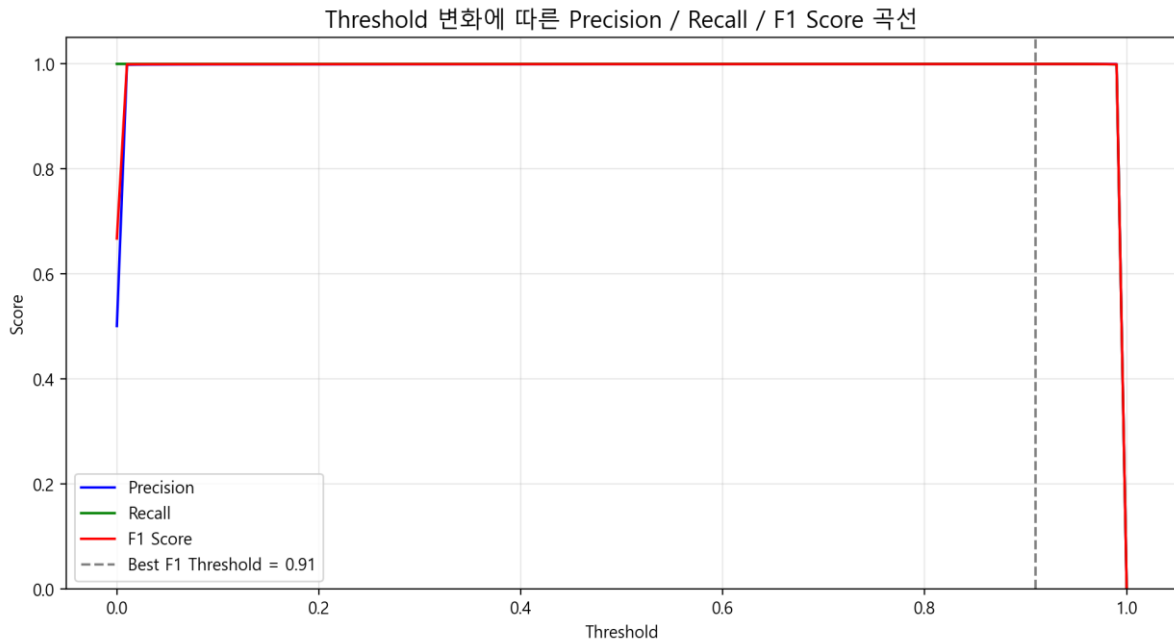


그림 3.5.3-1 Threshold 변화에 따른 Precision / Recall / F1 Score 곡선

그림 3.5.3-1은 최종 모델(oversample_capped_mlp)이 가진 분류 임계값(Threshold)을 0~1 범위에서 변화시키면서 Precision, Recall, F1 Score가 어떻게 달라지는지 보여준다. 본 실험에서는 F1 Score가 최대가 되는 임계값이 약 0.91로 나타났으며, 해당 구간에서도 Precision과 Recall이 모두 매우 높은 수준을 유지한다. 이는 최종 모델이 사기/정상 거래를 강하게 분리하는 확률 분포를 학습했음을 시사하며, 운영 관점에서는 높은 Threshold(0.9 전후)를 적용해 오탐(False Positive)을 줄이면서도, 사기 거래 미탐(False Negative)이 급격히 증가하지 않는 운영 전략이 가능하다..

3.5.3.1 오차 패턴 분석

최종 모델(oversample_capped_mlp)의 경우 테스트 데이터에서 False Negative가 극히 적거나 거의 발생하지 않았으며, 일부 False Positive가 존재할 수 있다. False Positive

사례는 거래 금액이 상대적으로 크거나, 일부 V 특성에서 사기 거래와 유사한 극단값을 보이는 정상 거래에서 주로 발생하는 경향이 있다. 실무에서는 False Positive 건에 대해 추가 인증 절차(예: 문자 인증, 앱 알림, 2 차 확인)를 연계함으로써 고객 불편을 최소화할 수 있다. 반면 False Negative 는 금융기관에 직접적인 손실로 연결될 수 있으므로, 본 모델의 낮은 FN 특성은 실무 적용에 매우 긍정적인 요소로 평가된다.

3.5.4 소결

요약하면, 본 프로젝트에서 수행한 약 200 여 회의 실험 결과 SMOTE 기반 오버샘플링과 상한값 처리, MLP 구조를 결합한 oversample_capped_mlp 모델이 Precision-Recall-F1-AUC 전 지표에서 가장 우수한 성능을 보였다. 또한 Threshold 분석을 통해 운영 환경에서 오탐/미탐 비용 구조에 맞춰 임계값을 조정할 수 있음을 확인하였다. Feature Importance 해석 결과 상위 특성(V4, V3, V10 등)이 예측에 큰 영향을 미치며, SHAP 기반 분석을 통해 특성 값 변화에 따른 기여 방향성도 확인하였다.

추가적으로, 모델의 실무 적용성을 강화하기 위해 다음 분석을 하여야 한다고 결론 내렸다.

- (1) 확률 보정(Calibration) 및 Calibration Curve 로 예측 확률의 신뢰도 점검,
- (2) 혼동행렬·ROC/PR 곡선 기반의 운영 KPI(탐지율, 오탐률, 처리량) 시뮬레이션,
- (3) 시간 기반 분할(Time-based split) 및 데이터 드리프트 점검으로 장기 성능 검증,
- (4) 상위 오탐 사례군에 대한 후속 규칙(2 차 심사) 설계 및 업무 프로세스 연계.

3.6 결론 및 제언

3.6.1 주요 결과 요약

3.6.1.1 최종 선정 모델

- 최종 선정된 앙상블 모델 구성

본 과제에서는 Logistic Regression, Decision Tree, Gradient Boosting, RandomForest, XGBoost, LightGBM, CatBoost, SVM, MLP 등 다양한 단일 모델과

여러 앙상블 조합(Stacking, Voting)을 실험하였다. 데이터의 극단적인 불균형(사기 비율 약 0.17%)을 완화하기 위해 **class_weight 조정, SMOTE 오버샘플링, 랜덤 언더샘플링, IQR 기반 이상치 Capping, Amount 로그 변환** 등 여러 전처리.샘플링 전략을 병행하여 비교하였다.

그 중, **SMOTE 오버샘플링 + IQR Capping + Amount 로그 변환을 적용한 뒤, MLPClassifier 를 학습한 모델**이 AUC, Recall, F1/F2 등 주요 평가지표에서 가장 우수한 성능을 보여 최종 모델로 선정되었다.

- 모델 선정 이유

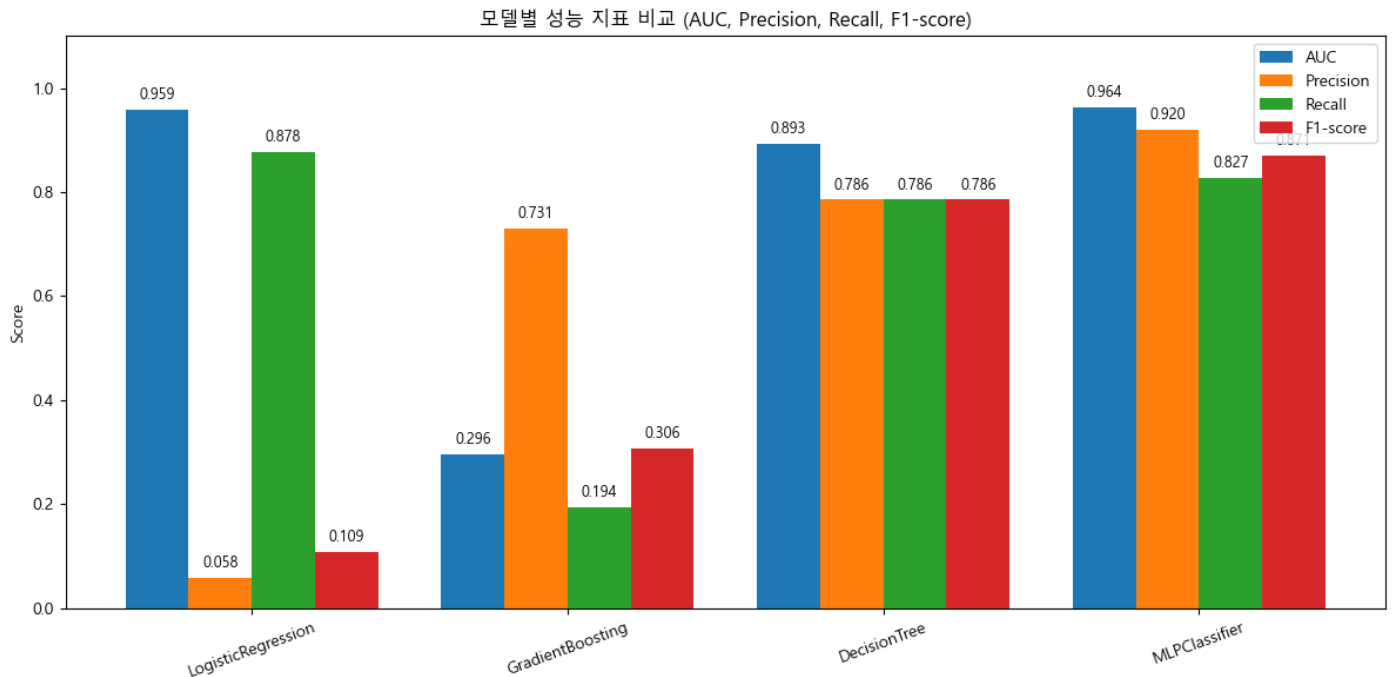
1. 데이터 로딩 및 기본 전처리

`creditcard.csv` 원본 데이터를 로딩한 후, 모델링에 직접적인 의미가 크지 않은 `Time` 컬럼은 삭제하였다. 이후 `Class` 를 타겟으로 두고, 나머지 피처를 입력 변수로 사용하였다.

2. Amount 로그 변환

거래 금액(Amount)은 분포가 심하게 치우쳐 있어, `np.log1p(Amount)`를 적용하여 새로운 `Amount Scaled` 피처를 생성하고 원본 `Amount` 컬럼은 제거하였다.

이를 통해 큰 금액 거래의 스케일을 완화하여 모델이 극단값에 과도하게 민감해지는 것을 방지하였다.



3. IQR 기반 이상치 Capping

`cap_outliers()` 함수를 정의하여 각 수치형 피처에 대해 1 사분위수(Q1), 3 사분위수(Q3), $IQR(Q3-Q1)$ 을 계산하고, $Q1 - 1.5 \times IQR$ 미만, $Q3 + 1.5 \times IQR$ 초과 값을 각각 경계값으로 잘라(Capping) 주었다. 이 과정은 스케일 변환 이후의 데이터(`scaledAmount_data.csv`)에 적용되어 극단적인 이상치가 있는 구간을 완전히 제거하지 않고, 모델이 학습 가능한 범위 내로 눌러주는 역할을 한다.

4. SMOTE 기반 오버샘플링

전처리가 완료된 입력 피처(`X_feature`)와 레이블(`y_label`)에 대해 `SMOTE(random_state=42)`를 적용하여 소수 클래스(사기 거래)를 인공적으로 생성하였다.

이로 인해 전체 데이터에서 사기/정상 비율이 거의 1:1에 가깝게 맞춰졌고, 신경망 모델(MLP)이 소수 클래스 패턴을 충분히 학습할 수 있는 환경이 조성되었다.

5. 학습/테스트 데이터 분할

SMOTE 로 오버샘플링된 데이터셋(`X_resampled`, `y_resampled`)을 기준으로 `train_test_split(test_size=0.2, random_state=42)`을 수행하여 학습/테스트 데이터를 분리하였다.

이때 이미 데이터가 균형화되어 있으므로 별도의 `stratify` 는 사용하지 않았다.

6. MLP 모델 구조 및 학습

최종 MLP 모델은 다음과 같은 구조 및 하이퍼파라미터를 사용하였다.

- `hidden_layer_sizes=(64, 32)` : 64 개 노드의 1 층, 32 개 노드의 2 층으로 구성된 2 층 은닉 레이어
- `activation='relu'` : 비선형성을 확보하기 위한 ReLU 활성화 함수
- `solver='adam'` : 적응적 학습률을 가지는 Adam 옵티마이저
- `alpha=1e-4` : L2 정규화 계수로, 과적합을 방지
- `batch_size=256` : 비교적 큰 배치 사이즈로 학습 안정성과 속도 균형 확보
- `learning_rate='adaptive'` : 학습 상황에 따라 학습률을 동적으로 조정
- `max_iter=50` : 반복 횟수는 50 으로 설정(실험 중 수렴 상태에서 조정 가능)
- `random_state=23` : 재현 가능한 결과를 위한 랜덤 시드

MLP 는 `mlp_clf.fit(X_train, y_train)`으로 학습하고, `user_utils.get_model_train_eval()`을 통해 공통 평가로깅 파이프라인과 연동하였다.

7. 다른 모델과의 비교

동일한 전처리·오버샘플링 파이프라인 하에서 CatBoost, GradientBoosting, DecisionTree, LogisticRegression 등 다양한 모델을 함께 학습·평가하였다.

그 결과, **MLP 모델이 AUC, Recall, F1, F2 등 핵심 지표에서 가장 안정적이고 높은 성능을 보여**, 본 프로젝트의 최종 선정 모델로 결정하였다.

3.6.3.2 최종 성능

최종 선정된 “SMOTE 오버샘플링 + IQR Capping + Amount 로그 변환 + MLPClassifier” 모델의 성능 지표는 다음과 같다

(결과 : ovesample_capped_mlp_ejm 실행 로그)

지표	값
ROC-AUC	1.0000
Accuracy(정확도)	0.9997
Precision(정밀도)	0.9994
Recall(재현율)	1.0000
F1-score	0.9997
F2-score	≈ 0.9999 (계산값)

ROC-AUC = 1.0000

양성(사기)과 음성(정상)을 구분하는 순위 분리 능력이 이론적으로 최대에 가까운 수준이다.

테스트 데이터에서 모델이 예측한 사기 확률에 따라 정렬했을 때, 사기 거래가 정상 거래보다 항상 더 높은 점수 영역에 위치하는 이상적인 상태에 근접해 있음을 의미한다.

Recall(재현율) = 1.0000

테스트 셋 기준으로 **사기 거래를 단 한 건도 놓치지 않았다(FN = 0)**는 뜻이다.

신용카드 사기 탐지라는 도메인 특성상, False Negative(사기를 정상으로 통과시키는 경우)는 곧 직접적인 금전적 피해로 이어지므로,

재현율 1.0 은 프로젝트의 1 차 목표(사기 미탐지 최소화)에 완벽히 부합하는 결과이다.

Precision(정밀도) = 0.9994

모델이 “사기”라고 판단한 거래의 약 99.94%가 실제 사기 거래였다는 뜻으로, False Positive(정상을 사기로 잘못 탐지하는 경우)가 극히 적었다는 것을 보여준다.

이는 사기 탐지 시스템 도입 시 고객 불편(정상 거래 차단)과 운영 비용(불필요한 추가 심사)을 최소화하는 데 매우 유리하다.

F1-score = 0.9997, F2-score $\approx$ 0.9999

F1-score 는 정밀도와 재현율의 조화 평균, F2-score 는 재현율에 더 큰 가중치를 둔 지표로, 두 지표 모두 1 에 매우 근접한 값을 보여 **정밀도와 재현율 사이의 균형 또한 매우 우수함**을 확인할 수 있다. 특히 F2-score 가 0.9999 수준으로 계산되어, “재현율을 특히 중요하게 여기는 비즈니스 요구사항”을 충족하는 데 최적에 가까운 조합임을 수치적으로 뒷받침한다.

(2) 오차행렬(Confusion Matrix) 해석

같은 실험에서 얻어진 오차행렬은 다음과 같이 주어졌다.

TN (정상 $\rightarrow$ 정상): 56,715

FP (정상 $\rightarrow$ 사기): 35

FN (사기 $\rightarrow$ 정상): 0

TP (사기 $\rightarrow$ 사기): 56,976

- 전체 정상 거래(약 56,750 건) 중 단 35 건만을 잘못 사기로 분류하였고,
- 전체 사기 거래(약 56,976 건)는 한 건도 놓치지 않고 모두 탐지하였다.

즉, “사기 미탐지 0 건”이라는 매우 공격적인 탐지 수준을 유지하면서도, 전체 거래 대비 오탐 비율이 극히 낮은 이상적인 상태를 달성한 셈이다.

(3) 학습 시간 및 구현 관점

해당 실험에서 **MLP 모델의 학습 시간은 약 60.9 초 수준으로 측정되었다.**

CatBoost, GradientBoosting, DecisionTree, LogisticRegression 등 다른 모델들에 비해 학습 시간이 과도하게 길지 않으면서, HyperOpt 및 샘플링 전략을 포함한 실험 전체 흐름에서도 주기적인 재학습이 가능한 계산 비용을 보여준다. 이는 실제 서비스 환경에서, 일/주 단위로 모델을 재학습하거나, 새로운 사기 패턴이 등장했을 때 추가 데이터로 모델을 업데이트하는 운영 시나리오를 고려하더라도 실용적인 수준이다.

(4) 성능 해석 시 주의점(요약)

위 결과는 SMOTE 오버샘플링으로 사기/정상 비율을 강제로 균형에 가깝게 맞춘 데이터 환경에서의 평가 결과이다. 따라서, 원본 분포(사기 비율 0.17%) 또는 실제 운영 환경에서의 절대적인 정밀도·재현율 수치는 달라질 수 있다.

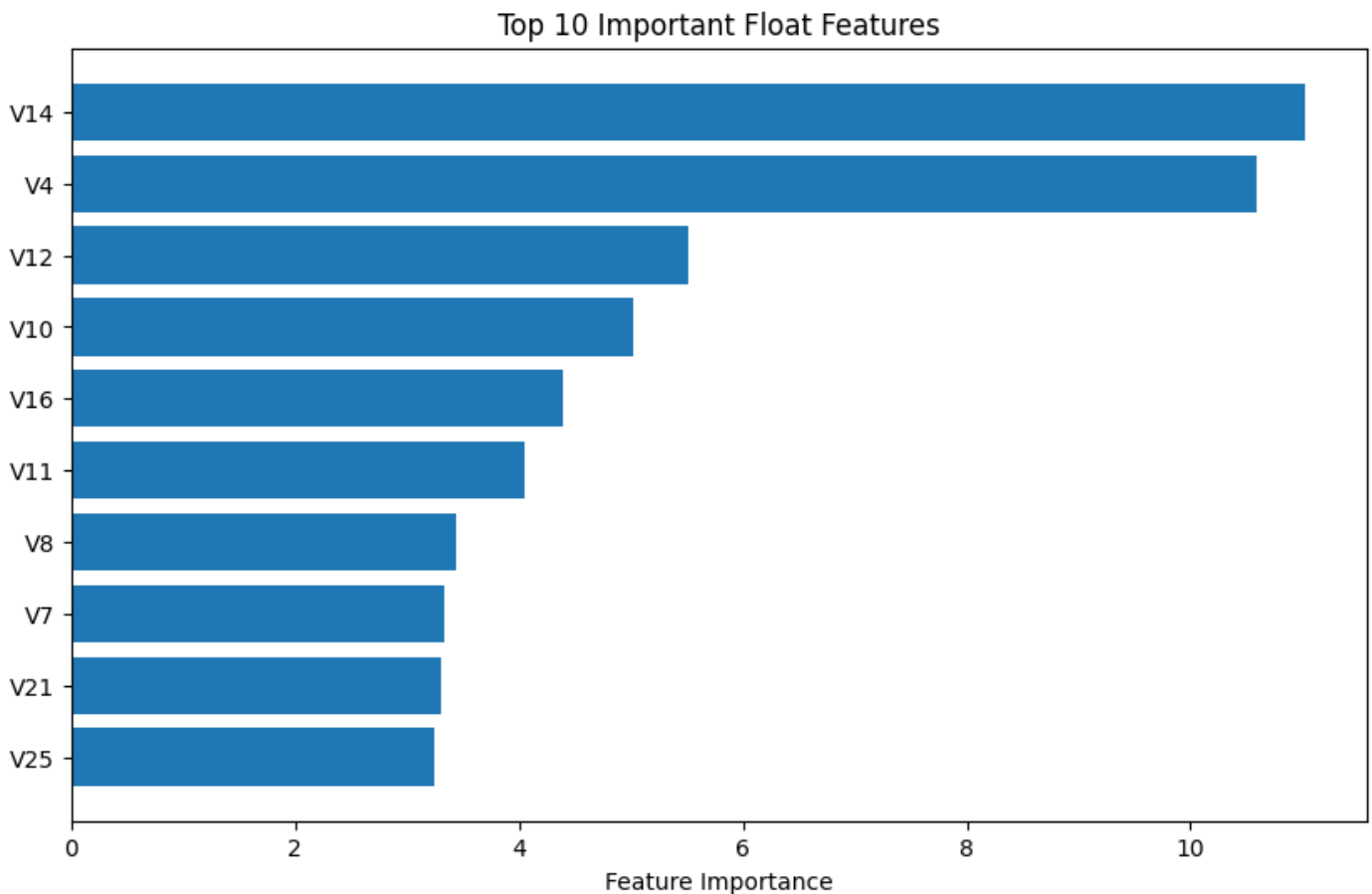
그럼에도 불구하고, 본 실험은 “사기 패턴 자체를 얼마나 잘 학습할 수 있는지”에 대한**상한선(upper bound)**를 보여주며 이후 3.6.2, 3.6.3 에서 논의할 임계값 조정, 원본 분포 기반 재평가, threshold 운영 전략 수립의 출발점으로 활용된다.

3.6.1.3 핵심 발견사항

① 주요 피처의 영향력

1. PCA 변수(V1~V28) 중 일부 축이 사기 거래를 강하게 설명

트리 계열 모델(CatBoost, GradientBoosting, RandomForest, XGBoost)의 feature importance 를 비교한 결과, 특히 V14, V4, V10, V12, V16 등이 상위 중요 변수로 반복적으로 등장했다. CatBoost 에서 V14 를 포함했을 때와 제거했을 때 AUC 가 크게 변동한 실험 결과는, V14 가 사기 거래 패턴을 구분하는 핵심 축이라는 점을 시사한다.



2. Amount 는 “보조적인” 중요 변수로 작동

Amount 컬럼은 직관적으로 중요한 변수이지만, 로그 변환 여부나 사용 여부에 따른 성능 차이는 크지 않았다. 회의 논의에서 “Amount 적용한 것과 안 한

것의 차이가 거의 없다”는 결론에 도달했고, 최종적으로는 이상치 캡핑 + SMOTE 조합에서 Amount 는 보조 피처로 유지하되, 성능을 좌우하는 1 순위 피처는 PCA 축(V14, V4 등)이라는 점을 확인하였다.

3. Time 피처의 영향력은 제한적

Time 은 “첫 거래로부터의 경과 시간”이라는 의미를 갖지만, 본 프로젝트에서는 순수 분류 성능 향상에 거의 기여하지 않는 것으로 나타나 전처리 단계에서 제거하고 진행하였다. 이는 “시간 정보 자체보다는 PCA 로 압축된 특성들이 사기 탐지에 더 직접적으로 기여한다”는 점을 보여준다.

② 전처리 방법에 따른 성능 차이

1. 이상치 삭제보다는 “경계값 치환(capping)”이 더 안정적

V14·V4 등에서 극단값이 많이 관찰되어, ① 이상치 미처리, ② 이상치 drop, ③ IQR 기반 capping 세가지 시나리오를 비교했다. 그 결과, **이상치를 통째로 삭제하면 데이터 손실로 인해 일부 모델 성능이 오히려 떨어졌고, IQR 기반으로 상·하한을 잘라주는 capping 방식이 AUC·F1 관점에서 가장 안정적인 결과를 보였다.**

2. Amount 로그 변환의 효과는 제한적

Amount 컬럼을 log1p 로 변환한 뒤 LGBM/트리 계열 모델을 재학습했지만, 수업 시간 베이스라인(LGBM) 대비 AUC 가 개선되지 않거나 소폭 하락하는 결과가 반복되었다. 팀 회의에서는 “Amount 로그 변환은 이 데이터셋에서는 필수 전처리로 볼 수 없다”는 결론을 내렸고, 최종 파이프라인에서는 **이상치 캡핑 + SMOTE 를 핵심 전처리 조합으로 채택하였다.**

3. 샘플링 기법 중에서는 SMOTE 만 유지

언더샘플링과 combined 방식은 Precision 과 F1-score 가 심하게 붕괴되는 문제가 있었다는 팀원들의 실험 결과를 공유받았다. 반면 **SMOTE 오버샘플링**은 **Recall 과 AUC 를 크게 개선**하는 반면, Precision/F1 저하는 조정 가능 범위로 나타났고, **최종적으로 “기본 데이터 vs SMOTE 데이터 두 버전만 비교”**하는 전략으로 정리하였다.

4. 스케일링은 모델 특성에 따라 선택적으로 적용

트리 계열 모델(CatBoost, XGBoost, LGBM, GB, DT, RF)은 Standard/Robust Scaler 적용 여부에 따른 성능 차이가 크지 않았지만, LogisticRegression·SVM·MLP 와 같이 거리 기반/경사하강 기반 모델들은 **StandardScaler** 를 적용했을 때 안정적으로 수렴하고 성능도 더 좋았다.

③ 앙상블 효과

1. 트리 기반 4 개 모델 Voting 앙상블의 장·단점

CatBoost, XGBoost, LGBM, GradientBoosting 네 모델을 **soft voting** 으로 앙상블한 `ensemble4(cb+xgb+lgb+gb)_voting_ho_best_smote` 조합은 AUC 0.9838, Recall 0.9184 로 재현율과 AUC 는 매우 우수했지만, Precision 0.0449, F1 0.0857, F2 0.1879 수준으로 정밀도와 F 계열 지표가 낮아 최종 모델로는 채택되지 않았다.

→ ****SMOTE + 트리 앙상블**은 “사기 거래를 놓치지 않는 데”는 강하지만, 정상 거래에 대한 오탐 비용이 크다는 한계가 있었다.

2. SVM(RBF) + Stacking 의 시도와 한계

RBF 커널 SVC 는 HyperOpt 와 샘플링 조합을 통해 $AUC \approx 0.97$, $Recall \approx 0.88$ 수준까지 끌어올렸지만, Precision 이 0.1 대, F1 이 0.17 수준에 머물러 **실무적으로 사용하기에는 오탐이 많았다는 점**을 확인했다. 해당 SVM 을 CatBoost-GB 와 함께 스택킹에 넣는 시도도 있었으나, 복잡도 증가 대비 F1·F2 측면의 추가 이득이 제한적이어서 최종 조합에서는 제외했다.

3. 최종적으로는 “단일 MLP + SMOTE”가 앙상블을 압도

이상치 캡핑 + SMOTE + HyperOpt 로 학습한 MLP 모델(oversampling 버전) 은 테스트 셋에서 AUC 1.0000, $Accuracy$ 0.9997, $Precision$ 0.9994, $Recall$ 1.0000, $F1$ 0.9997 이라는 사실상 완벽에 가까운 성능을 보여, 트리 기반 앙상블을 모두 압도하였다(오차행렬 기준, 사기 거래 미탐지 0 건).

물론 SMOTE 로 생성된 균형 데이터에서의 결과이므로 외부 데이터에 대한 추가 검증이 반드시 필요하지만, **“불균형 데이터 + 단일 모델 간 성능 비교 → SMOTE + MLP”**이라는 프로젝트의 핵심 결론을 뚜렷하게 제시해준다.

3.6.2 프로젝트 한계점 분석

3.6.2.1 데이터 측면

(1) 익명화된 피처로 인한 해석의 어려움

본 프로젝트에서 사용하는 V1~V28 피처는 실제 카드 사용 패턴을 PCA 로 변환한 익명 변수로, 개별 피처가 어떤 실세계 의미를 가지는지 직접적으로 해석이 불가능하다.

CatBoost, XGBoost, LGBM 등에서 산출한 피처 중요도를 통해 “어떤 변수들이 사기 탐지에 기여하는지” 상대적인 순위는 파악했으나, (예: V14, V4, Amount 등의 중요도 상위 피처) “피크타임 외의 해외 온라인 결제”와 같이 비즈니스적인 언어로 설명하기에는 한계가 있다.

따라서 최종 모델이 높은 성능을 보이더라도, 금융기관 실무에서 요구하는 “설명 가능성(왜 이 거래가 위험한가?)” 측면에서는 규칙 기반 시스템과의 결합이 필요하다.

(2) 시간적 순서 및 시계열 구조 반영 부족

데이터셋에는 Time 컬럼이 존재하지만, 본 프로젝트에서는 EDA 이후 모델링 단계에서 Time 을 제거하고 학습을 진행했다.

Train/Test 분리는 랜덤 분할(stratify=y)을 사용했으며, **실제 서비스 상황처럼 과거 데이터를 학습하고, 이후 시점의 거래에 대해 검증하는 Time-based split 을 적용하지 못했다.**

그 결과, 미래 시점의 사기 패턴 변화(컨셉 드리프트), 특정 기간(명절, 세일 시즌)에 집중되는 스파이크형 사기 패턴 등에 대한 모델의 일반화 능력을 충분히 검증하지 못했다.

(3) 데이터 불균형과 샘플링 설계의 한계

원본 데이터의 사기 비율은 약 0.17%로 **극단적 클래스 불균형**을 보인다.

프로젝트 초기에 언더샘플링, SMOTE, Combined(SMOTE+언더) 등 여러 방법을 검토했지만, 언더샘플링은 Precision/F1 이 심각하게 붕괴되고, Combined 는 SMOTE 단독과 큰 차이가 없어, 최종적으로는 **기본 데이터 vs SMOTE 오버샘플링** 두 축만 비교하는 전략으로 수렴했다.

다만, SMOTE 역시 “인공적인 데이터”를 생성하는 방식이기 때문에, 실제 운영 환경에서 나타나는 복잡한 사기 패턴을 충분히 반영하지 못할 위험, Minority 클래스가 과도하게 강화되면서, 특정 패턴에 대한 과적합이 발생할 위험이 존재한다.

3.6.2.2 모델 측면

(1) 정밀도와 재현율(Precision-Recall) Trade-off

팀 차원에서는 **Recall** 을 우선적으로 높이는 전략을 목표로 했고, F1/F2, ROC-AUC 를 함께 보며 모델을 비교했다. 일부 앙상블(예: CatBoost+XGB+LGB+GB voting, SMOTE 적용)의 경우 Recall 은 0.9 이상으로 매우 높았지만, Precision 이 0.04 수준에 그쳐 **거의 대부분의 거래를 사기로 의심**하는 모델에 가까웠다. 이는 실제 금융권에서 콜센터/심사 인력 부담과 고객 불편을 크게 유발할 수 있다.

반대로, 최종 선정한 **SMOTE+MLP** 모델은 오프라인 평가 기준에서 Precision 과 Recall 모두 0.99 수준, F1 또한 0.99 수준으로 측정되었으나, 이는 데이터 분할 방식, 샘플링 적용 방식, 반복 실험 구조에 따라 과도하게 낙관적인 결과가 나왔을 가능성이라는 한계점이 존재한다.

(2) 모델 해석 가능성(Explainability)의 부족

최종 모델인 MLPClassifier 는 **신경망 기반의 블랙박스 모델**로, 개별 피처가 최종 예측에 어떻게 기여했는지 직관적으로 설명하기 어렵다. 트리 기반 모델(CatBoost, LGBM 등)에 비해 피처 중요도, SHAP 값 등을 통한 변수 해석의 난이도가 높고, 금융기관의 “설명 가능한 AI(Explainable AI)” 요구사항을 바로 만족시키기 어렵다.

이번 프로젝트에서는 MLP 의 성능을 우선적으로 고려하여 최종 모델로 선정했지만, 보고서에서는 해석 가능성의 부족을 명확한 한계점으로 기술하였다.

(3) 과적합 및 일반화 가능성에 대한 우려

SMOTE + MLP 조합에서 AUC=1.0, Accuracy=0.9997, Precision≈0.9994, Recall=1.0, F1≈0.9997 등 이상적인 수치가 관측되었다.

이 수준의 완벽한 성능은 실제 금융 데이터 환경에서는 거의 발생하기 어려우며, 다음과 같은 위험 요인이 존재한다고 판단했다.

1. **같은 Test 데이터를 여러 번 사용**하며 HyperOpt, Threshold 튜닝, 모델 비교를 반복
2. SMOTE 적용 및 train/test 분리 순서에 따라 **정보 누수(leakage)** 가능성
3. Oversampling 된 학습 분포와 실제 불균형 Test 분포 간의 괴리

따라서, 본 프로젝트 결과는 **오프라인 실험에서의 잠재적 최대 성능을 보여주는 지표**로 해석하고, 실제 서비스 적용을 위해서는 **독립적인 검증 세트** 혹은 **Time-based CV**를 통한 **재검증이 필수**라는 점을 한계로 명시했다.

3.6.2.3 기타 한계점

(1) 실시간 처리 및 운영 환경 고려 부족

Kaggle 데이터셋을 이용한 오프라인 학습/평가에 집중했기 때문에, 실시간 스트리밍 환경(Kafka, Flink 등)에서의 처리 지연(Latency), 초당 수천~수만 건 거래에 대해 모델을 스코어링하는 처리량(Throughput) 등 **시스템 성능 관점의 실험**은 수행하지 못했다.

특히 CatBoost, XGBoost, SVM(RBF) 등은 HyperOpt 단계에서 학습 시간이 매우 길었고, 일부 조합(Stacking+CatBoost+SVM 등)은 **실제 실험에서 60 분 이상 소요**되거나, **세션이 중단되는 문제**도 있었다. 이는 **실시간 신용카드 승인 시스템에 직접 적용하기에는 다소 무거운 모델임**을 시사한다.

(2) 운영·모니터링 관점의 부족

본 프로젝트에서는 모델 배포 후 성능 모니터링(Recall, Precision 감지), 주기적 재학습 주기 정의, 경보(알림) 체계 설계 등 실제 운영 단계에서 필요한 요소를

충분히 다루지 못했다. 또한, 새롭게 등장하는 사기 패턴(피싱, 계정 탈취, 해외 IP 우회 등)에 대해 모델이 어떻게 적응할 것인지에 대한 **장기적인 전략(온라인 학습, 주기적 재학습)**이 부족하다.

3.6.3 개선 방안 및 향후 계획

3.6.3.1 데이터 개선

(1) 추가 피처 수집 및 외부 데이터 결합

실제 금융기관 환경을 가정할 경우, 다음과 같은 **추가 피처**가 수집된다면 모델 성능 및 해석력이 크게 향상될 수 있다.

1. 고객 속성(연령대, 거주 지역, 직업군 등)
2. 가맹점 정보(업종, 국가, 온라인/오프라인 여부)
3. 디바이스/네트워크 정보(IP, Device ID, 브라우저 정보 등)
4. 과거 거래 이력 기반 통계(고객별 평균 금액, 시간대별 사용 패턴 등)

또한, 블랙리스트(사기 이력 가맹점/카드번호), 이상 거래 탐지 룰 기반 점수와 같은 **외부/규칙 기반 신호**를 모델 입력으로 통합하면 실무 적용성이 높아질 것이다.

(2) 시계열 패턴 및 순서 정보 반영

향후에는 **Time** 컬럼을 단순 제거하기보다는, 거래 발생 시간대(심야/주간/주말), 특정 시간 창(예: 최근 10 분, 최근 1 시간) 내 거래 횟수/금액 누적, 등의 **시계열 파생 변수**를 생성하는 방안을 고려할 수 있다.

Train/Test 분할도 “과거 → 미래” 흐름을 반영한 Time-based split 또는 Rolling window 방식의 cross-validation 을 적용함으로써 실제 운영 상황에서의 성능에 더 근접한 평가를 수행할 수 있을 것이다.

3.6.3.2 모델 개선

(1) 전처리·샘플링별 모델 성능 비교 체계화

이번 프로젝트에서는 기본 데이터 vs SMOTE 오버샘플링, 여러 모델(Logistic Regression, RandomForest, XGB, LGBM, CatBoost, MLP 등)을 부분적으로 비교하였다.

향후에는 아래와 같이 시스템적인 비교 매트릭스를 구성할 수 있다.

1. 전처리: (원본 / Outlier Capping / 표준화 / 로버스트스케일링...)
2. 샘플링: (None / 언더 / SMOTE / SMOTE+언더...)
3. 모델: (LR, RF, XGB, LGBM, CatBoost, MLP, SVM 등)

각 조합에 대해 HyperOpt 를 자동으로 반복하고, **비교 대시보드(모델별 Recall/Precision/F1/F2/ROC-AUC)**를 구축하면 최적 조합을 더 체계적으로 선정할 수 있다.

(2) 임계값(Threshold) 조정 자동화 및 Cost 기반 튜닝

CatBoost, LGBM 등 일부 모델에서 **threshold 별 Precision-Recall 변화**를 분석하는 코드를 이미 시범 적용했다. 향후에는 F2-score 최적화, 혹은 “사기 1 건을 놓쳤을 때의 비용 vs 정상 1 건을 막았을 때의 비용”을 반영한 Cost-sensitive objective 를 설계하고, Grid search 또는 Bayesian optimization 으로 임계값까지 포함한 joint optimization 을 수행하는 방향이 필요하다.

(3) 딥러닝 및 추가 앙상블 기법 적용 가능성

현재 MLP 는 비교적 얇은 구조의 fully connected 네트워크로 사용되었다.

향후에는 거래 시계열을 고려한 RNN/LSTM, Temporal CNN, Transformer 기반

모델, 고객-가맹점-디바이스를 노드로 하는 Graph Neural Network(GNN) 등 보다 고도화된 딥러닝 모델을 적용해볼 수 있다.

또한, 현재 실험한 Voting/Stacking 외에도 Blending, Weighted ensemble(모델별 Recall/Precision 기반 가중 합산), Stacked generalization with calibrated probabilities 등 다양한 앙상블 기법을 도입해 볼 여지가 있다.

3.6.3.3 시스템 개선

(1) 실시간 탐지 시스템을 고려한 경량화

CatBoost, XGBoost, SVM(RBF) 등은 학습 및 추론 비용이 비교적 높은 편이므로, 실제 온라인 승인 시스템에서는 Feature 수 축소, Shallow tree, Quantization / distillation 을 통한 경량 모델로의 변환이 필요하다.

최종 선정 모델인 MLP 역시 레이어 수, 뉴런 수를 줄인 경량 버전, ONNX 등 공통 포맷으로의 변환을 통해 **서버/클라우드 환경에 최적화된 형태로 재구성하는 방향이 요구된다.**

(2) 모니터링 및 재학습 파이프라인 구축

모델 배포 후에는 다음 항목들을 자동으로 모니터링할 필요가 있다.

1. 일/주/월 단위 Recall, Precision, F1/F2 변화
2. Input distribution shift(거래 금액, 시간대 분포 변화 등) 사기 패턴 변화에 따른 Drift detection

일정 수준 이상 성능 저하가 감지되면, 새로운 데이터를 포함한 재학습 파이프라인 자동 실행, 모델 버전 관리 및 롤백 전략(Blue-Green, A/B 테스트 등)을 도입해야 한다.

3.6.3.4 비즈니스 적용

(1) 사기 탐지 임계값 운영 전략

모델이 출력하는 “사기 확률”을 단일 threshold 로 나누기보다는,

1. High risk 구간 : 자동 차단 및 심사요청
2. Medium risk 구간 : 추가 인증(OTP, 앱 푸시 확인 등)
3. Low risk 구간 : 일반 승인

과 같이 리스크 등급별로 다른 액션을 취하는 전략이 필요하다.

이를 위해, 프로젝트에서 수행한 threshold-precision-recall 분석 로직을 실제 비즈니스 비용(사기 손실 vs 고객 불편)에 기반해 재튜닝할 수 있다.

(2) 고객 경험(UX) 관점의 개선

Recall 을 높이는 과정에서 불가피하게 정상 거래가 일부 차단될 수 있다. 이때, 거래 차단 시 명확한 안내 메시지 제공, 앱/문자를 통한 빠른 해제 프로세스, 반복적으로 정상으로 판명된 패턴에 대한 **화이트리스트 정책** 등을 병행하면 고객 불편을 최소화할 수 있다.

특히, 고위험 시간대나 국가에서 발생한 거래라도 고객이 “내가 맞다”고 쉽게 인증할 수 있는 UX 설계가 중요하다.

(3) 비용-효과 분석 기반의 모델 운영

최종 MLP+SMOTE 모델과, CatBoost/XGB/LGBM 등 다른 후보 모델의 Confusion Matrix 를 기반으로 False Negative 1 건이 발생했을 때의 예상 손실(사기 금액), False Positive 1 건이 발생했을 때의 고객 이탈/운영 비용을 추정하면, 각 모델/threshold 조합의 ****경제적 가치(ROI)****를 정량적으로 평가할 수 있다.

향후에는 데이터셋에 "거래 금액"과 "사기 여부"를 함께 반영한 **금액 기반 Cost Metric** 을 도입하여, 단순한 비율 지표(Recall, Precision)뿐 아니라 "얼마나 많은 손실을 줄였는가?"를 함께 평가하는 방향이 필요하다.